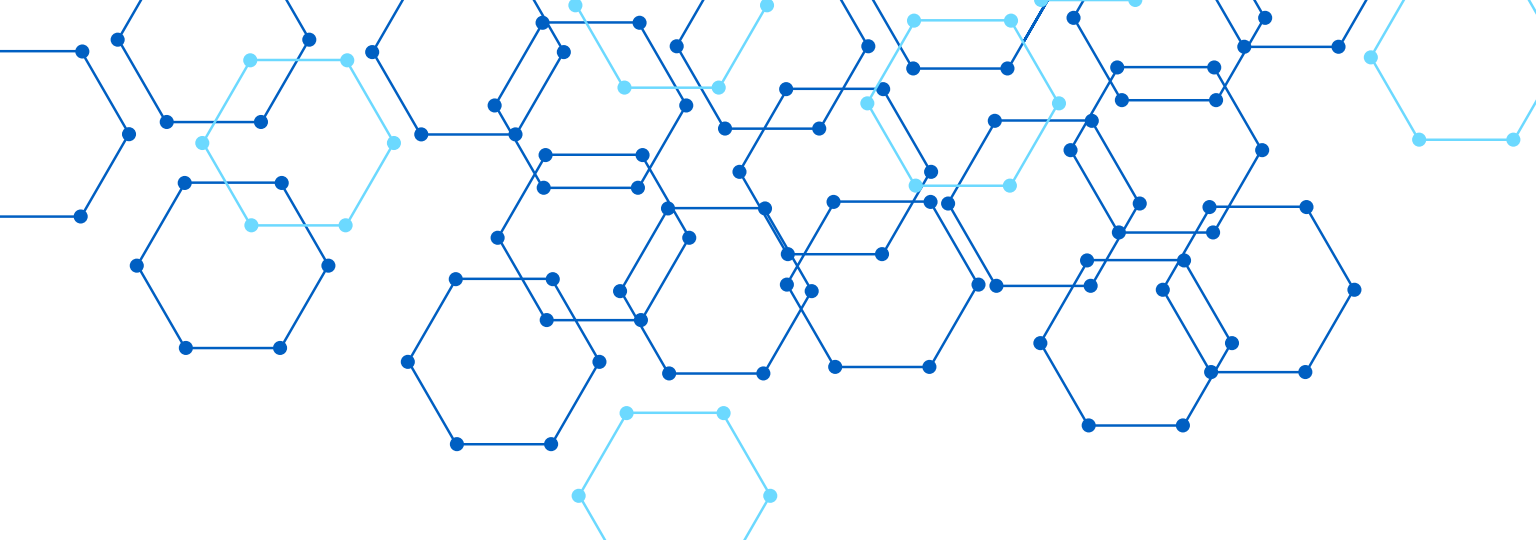
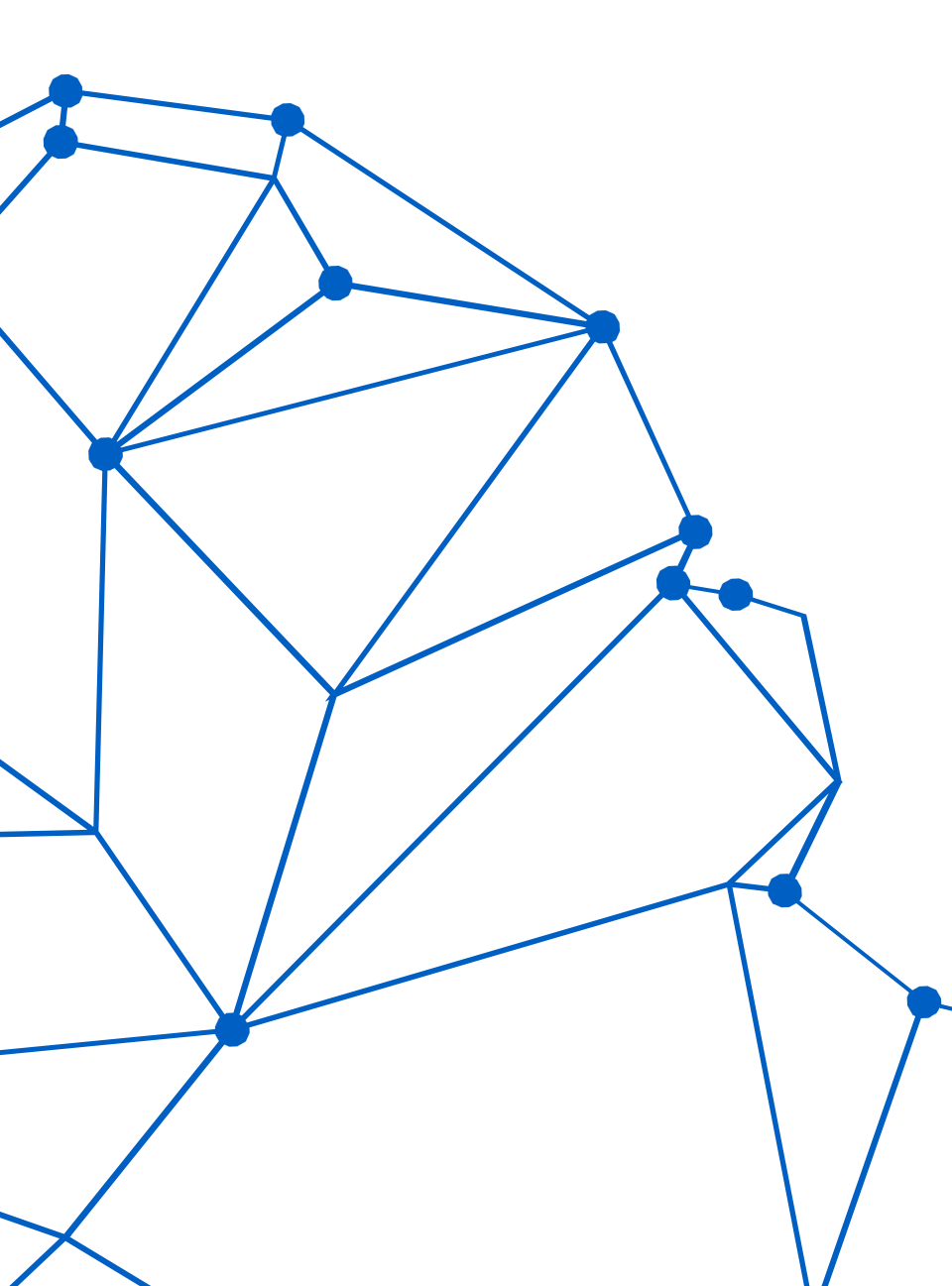
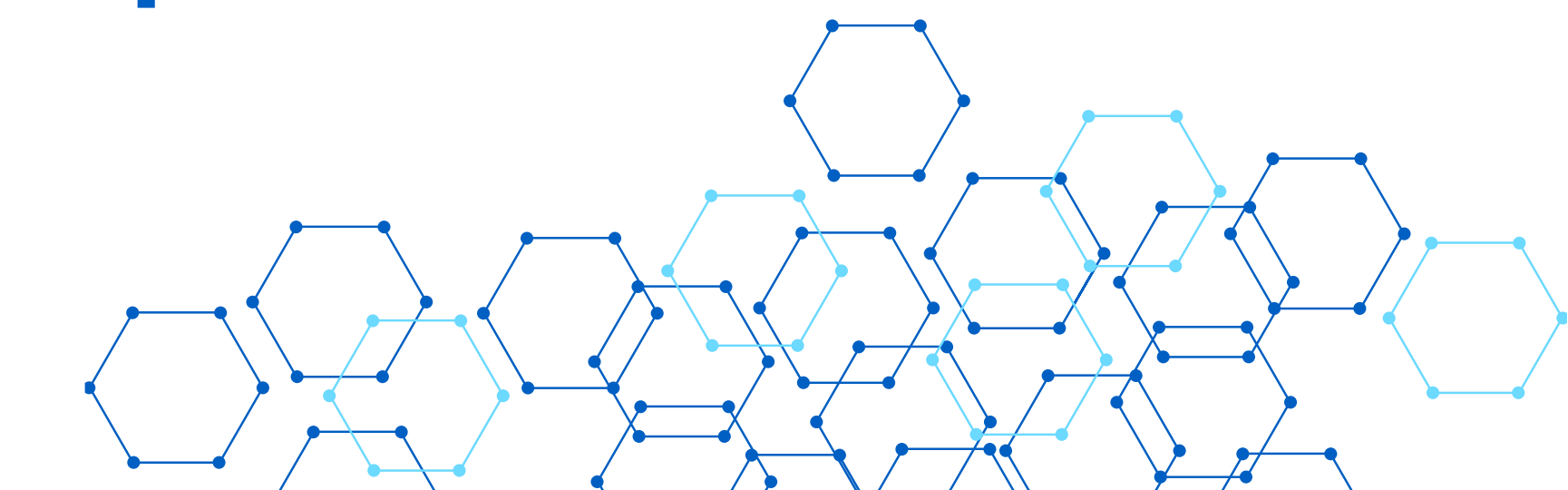


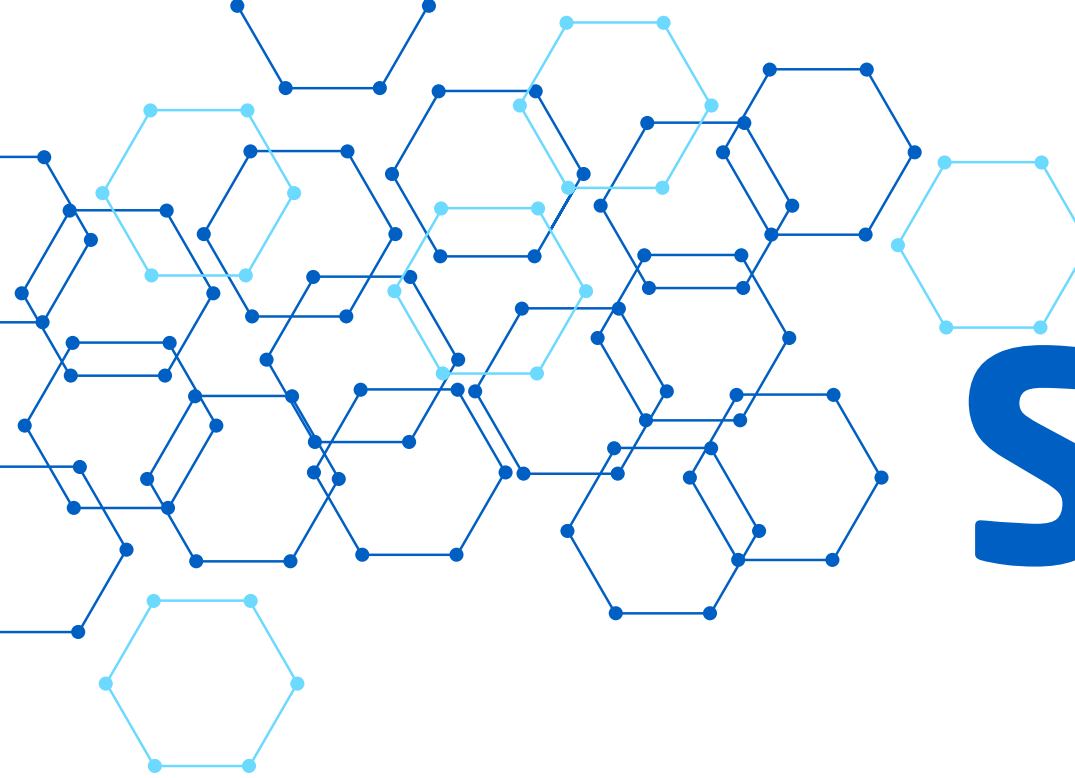


THE SOFTWARE PROCESS



Presented by Group 1 BSIT-S-2A

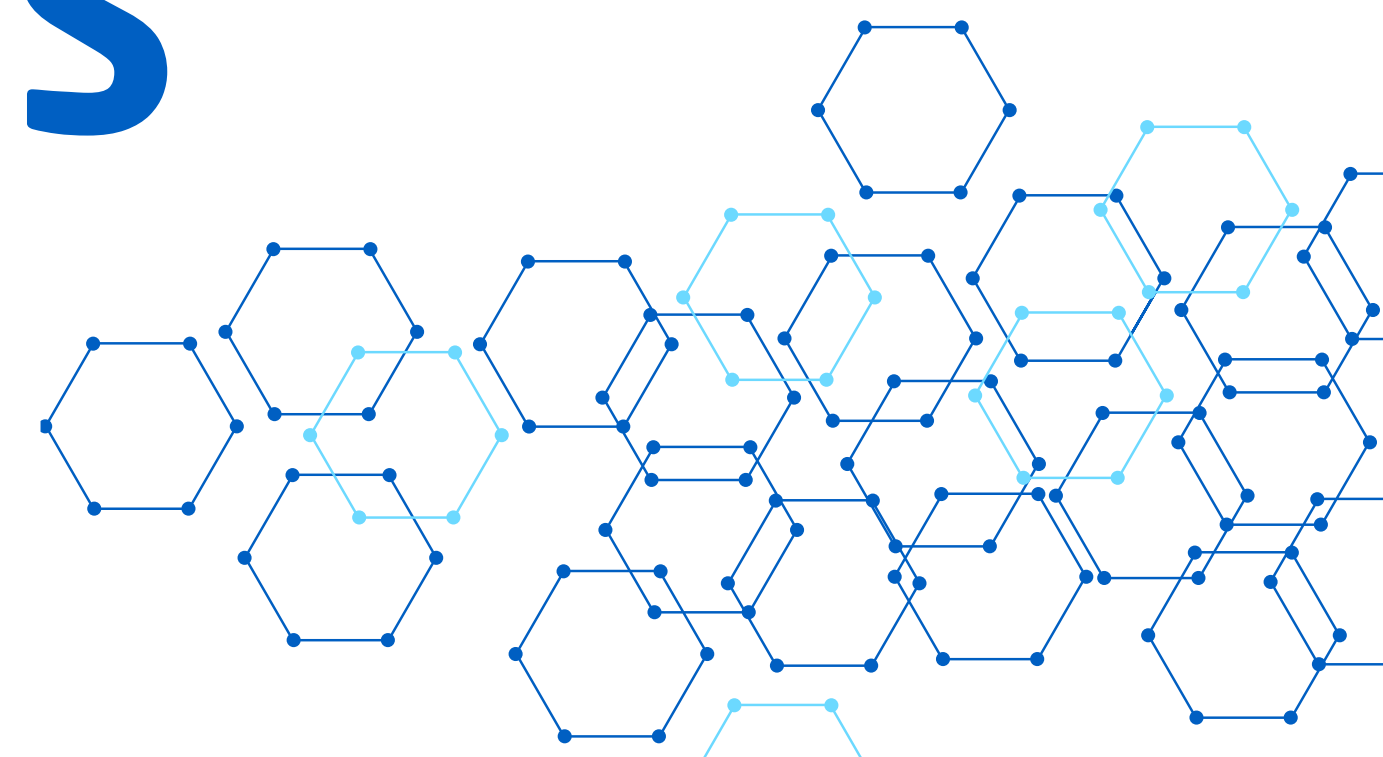


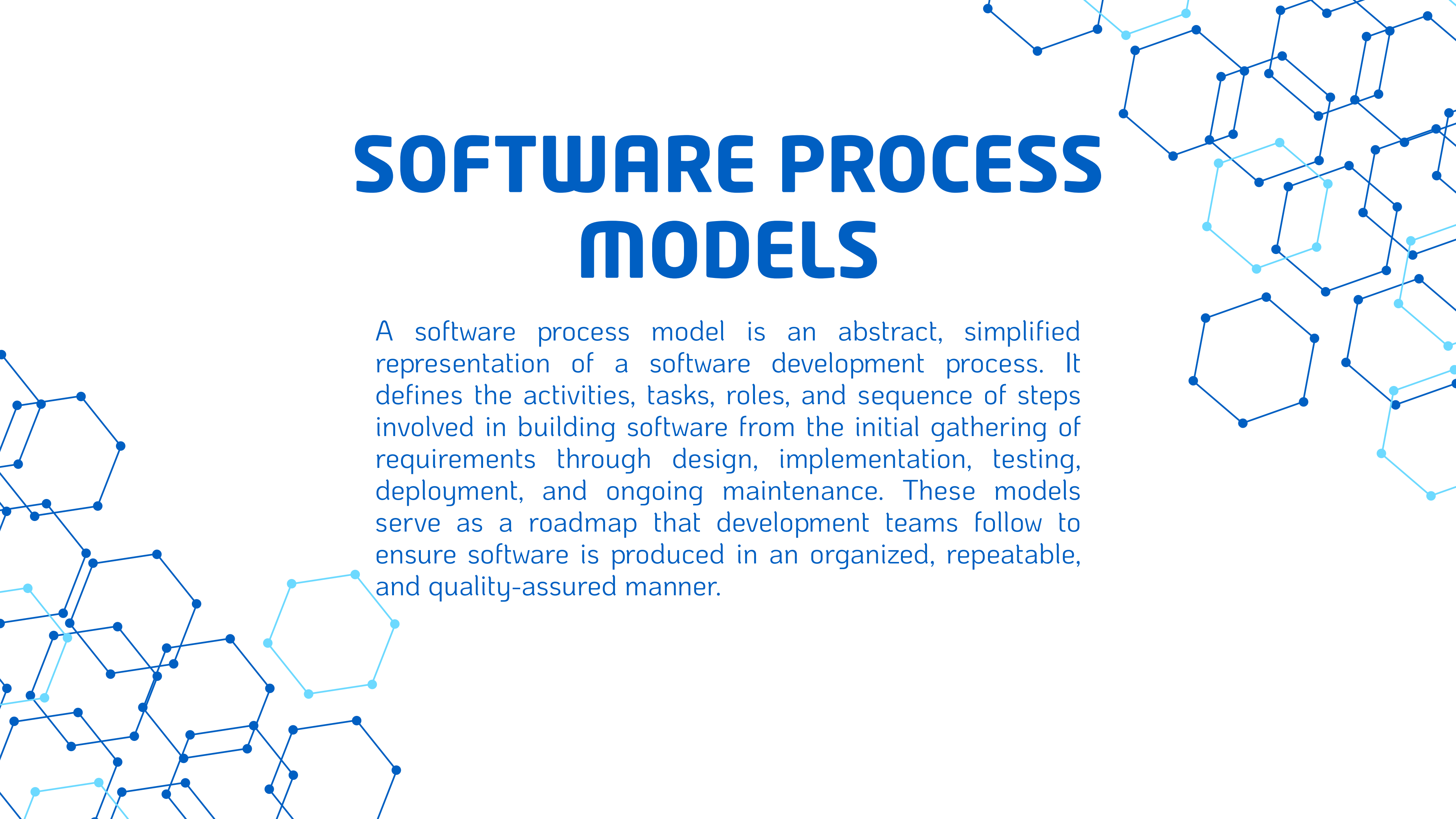


SOFTWARE PROCESS MODELS



REPORTER:
YUNICE MATABILE





SOFTWARE PROCESS MODELS

A software process model is an abstract, simplified representation of a software development process. It defines the activities, tasks, roles, and sequence of steps involved in building software from the initial gathering of requirements through design, implementation, testing, deployment, and ongoing maintenance. These models serve as a roadmap that development teams follow to ensure software is produced in an organized, repeatable, and quality-assured manner.



PURPOSE

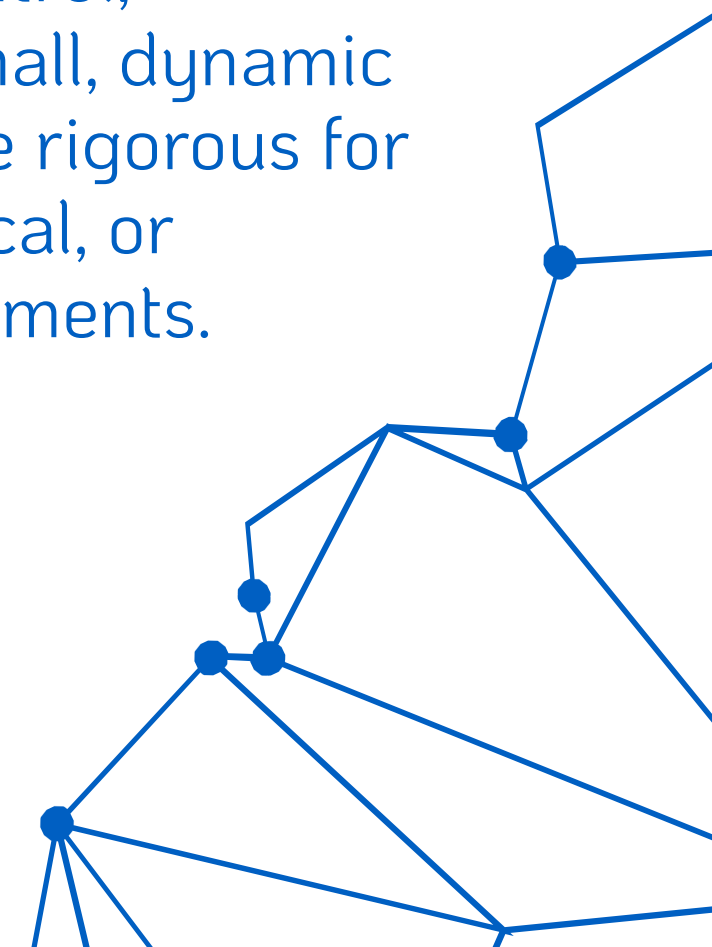
Provides a structured framework to plan, control, and deliver software systematically by defining clear phases, roles, and milestones. This helps teams coordinate efforts, track progress, and maintain alignment with business goals throughout the development lifecycle.

BENEFITS

Improves predictability, reduces risk, and ensures consistent quality across projects. Standardized processes enable early issue detection, better resource planning, and repeatable success patterns, while minimizing scope creep and integration failures.

USAGE

Chosen based on project size, team structure, requirements clarity, and client needs. The right methodology balances flexibility with control, lightweight for small, dynamic projects and more rigorous for large, safety-critical, or regulated environments.



WATERFALL MODEL

Linear, sequential phases:
Requirements → Design →
Implementation → Testing →
Maintenance. Best for well-
defined, stable requirements.

INCREMENTAL MODEL

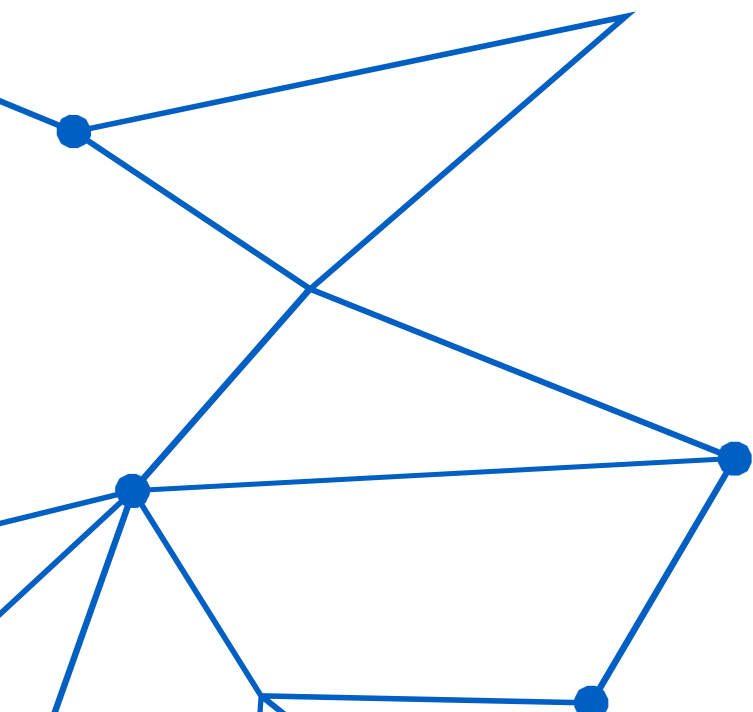
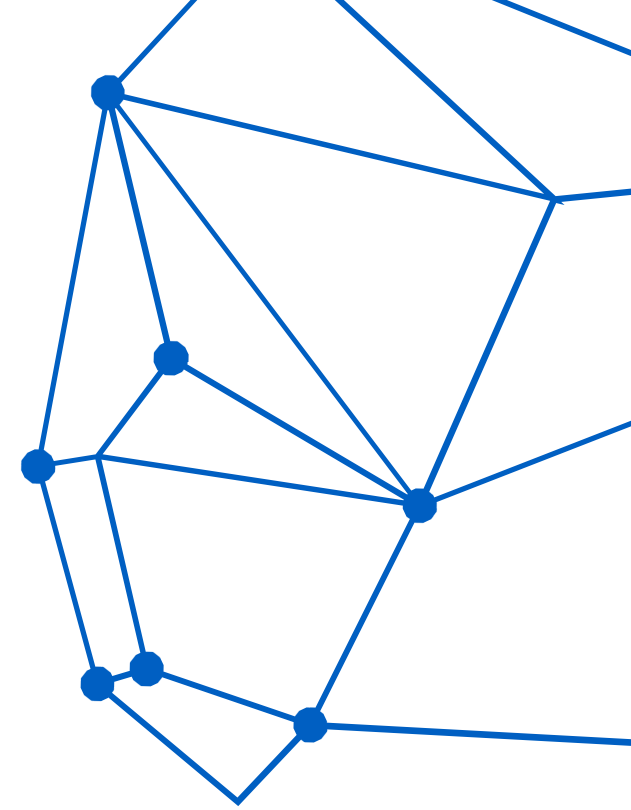
Software is built and
delivered in increments.
Each increment adds
functionality, allowing early
delivery and user feedback.

SPIRAL MODEL

Combines iterative
development with risk
analysis. Each spiral loop
covers planning, risk
assessment, engineering, and
evaluation.

AGILE / SCRUM

Iterative, collaborative, and
adaptive. Focuses on working
software, customer
collaboration, and responding
to change.



WATERFALL MODEL



TECHNOLOGICAL UNIVERSITY OF THE PHILIPPINES

Students Access Module

User Authentication

Username:

Password:

Birthdate:

Clear Entries

Login

Forgot your password? [Click here](#)

INCREMENTAL MODEL



SPIRAL MODEL

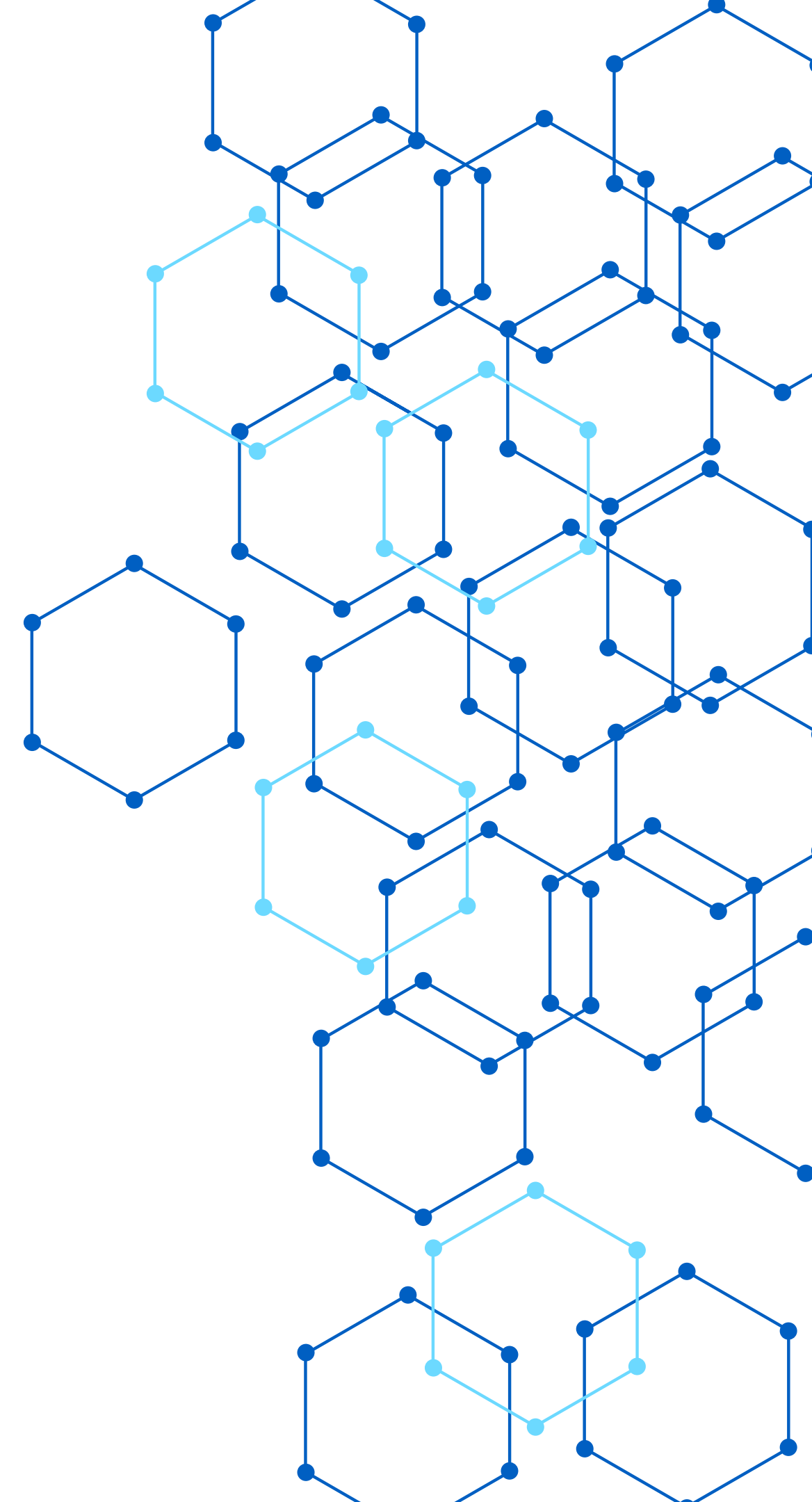


AGILE / SCRUM



CHARACTERISTICS OF A GOOD PROCESS MODEL

A good software process model must be clear and understandable by the whole team. It should be flexible enough to adapt to changes without disrupting the entire workflow. It must be measurable so teams can track progress at every stage. It should support communication between developers, managers, and clients. Finally it must be repeatable so the same process can be applied across different projects and still produce consistent and reliable results.



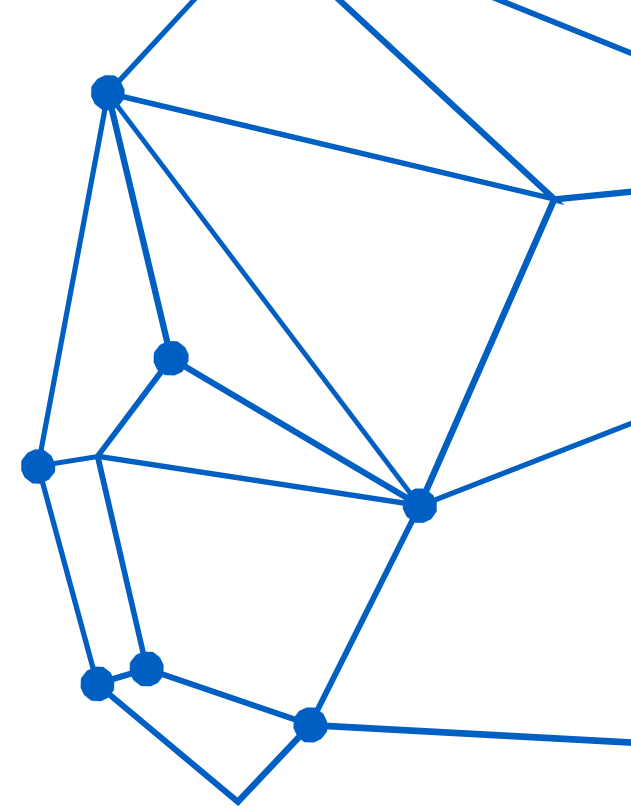
ADVANTAGES AND DISADVANTAGES OF PROCESS MODELS

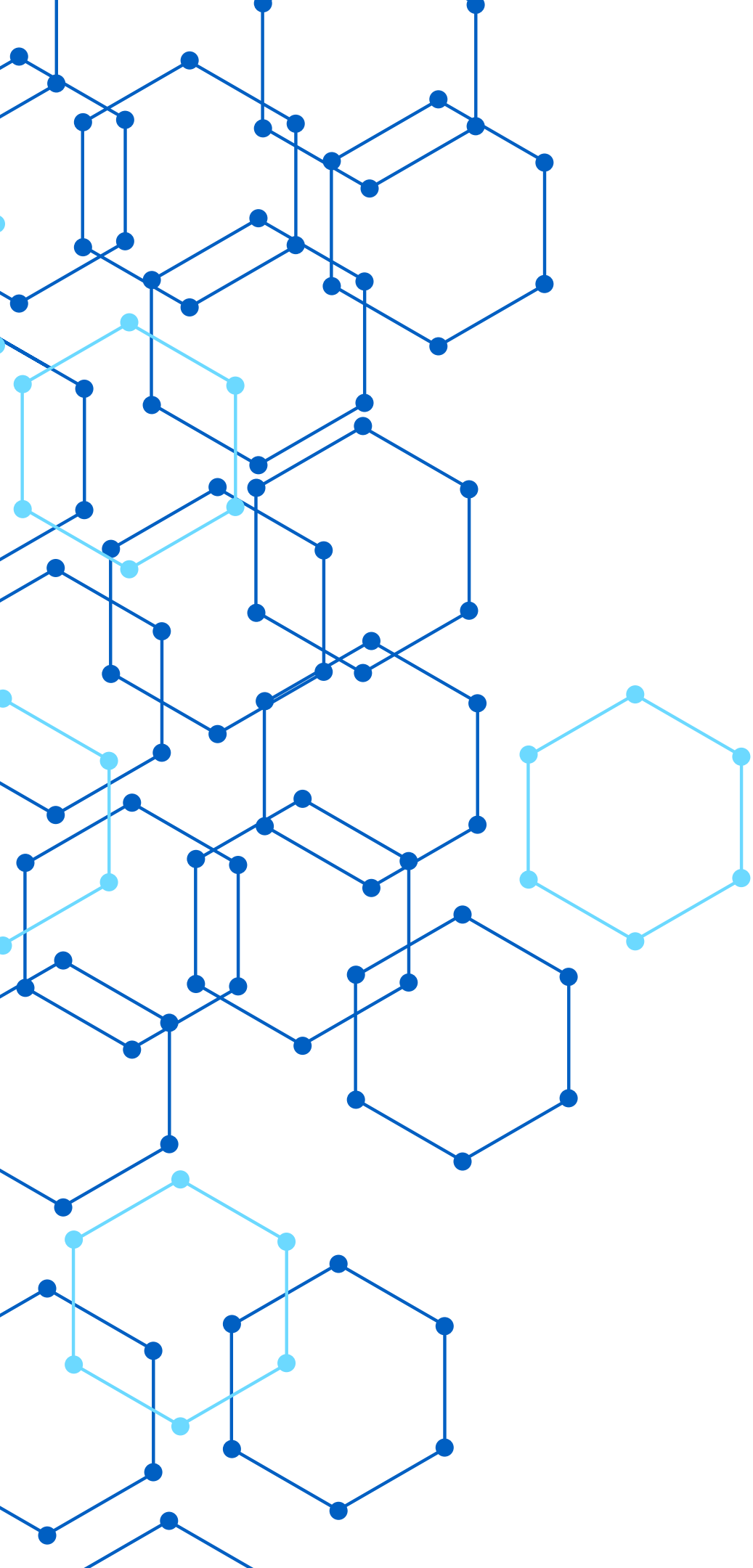
ADVANTAGES

Brings structure and discipline to development. Improves team coordination and makes timelines more predictable. Helps identify risks early. Provides clear documentation useful for onboarding and long term maintenance.

DISADVANTAGES

Some models like Waterfall are too rigid for changing requirements. Agile can be difficult to manage in large teams without strong discipline. Models can sometimes slow down development through excessive documentation or procedures.





HOW TO CHOOSE THE RIGHT PROCESS MODEL?

If requirements are clear and fixed, use Waterfall. If early delivery and ongoing feedback are needed, use the Incremental model. If the project involves high risk and uncertainty, use the Spiral model. If the team needs speed and flexibility, use Agile. Some teams combine elements of multiple models, known as a hybrid approach.

OVERVIEW: THE 4 BASIC PROCESS ACTIVITIES

1. Specification

2. Development

3. Validation

4. Evolution

- ORGANIZED SEQUENTIALLY IN THE WATERFALL MODEL
- ORGANIZED AS INTERLEAVED STEPS IN INCREMENTAL DEVELOPMENT
- HOW ACTIVITIES ARE CARRIED OUT DEPENDS ON: TYPE OF SOFTWARE, THE TEAM, AND ORGANIZATION
- EACH ACTIVITY PRODUCES IMPORTANT OUTPUTS USED IN THE NEXT STAGE
- DIFFERENT PROJECTS MAY ORGANIZE THESE ACTIVITIES DIFFERENTLY

1. *SOFTWARE SPECIFICATION*

Understanding WHAT the system should do and identifying CONSTRAINTS.

4 ACTIVITIES IN REQUIREMENTS ENGINEERING

1

FEASIBILITY STUDY – IS IT POSSIBLE AND COST-EFFECTIVE TO BUILD?

1

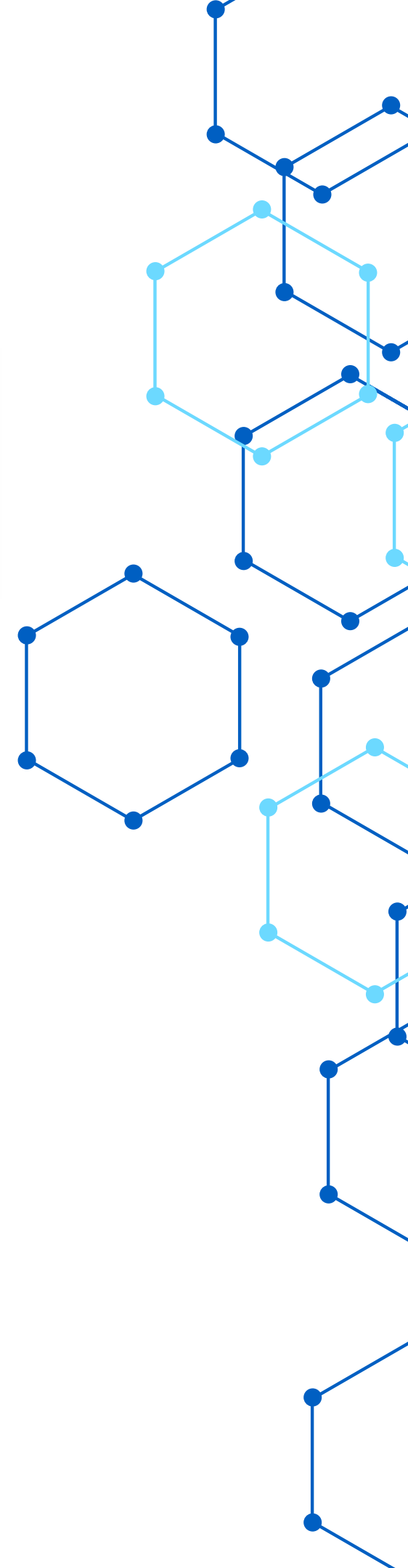
ELICITATION & ANALYSIS – GATHER AND UNDERSTAND USER NEEDS

1

REQUIREMENTS SPECIFICATION – DOCUMENT THE REQUIREMENTS CLEARLY

1

REQUIREMENTS VALIDATION – CHECK FOR CORRECTNESS AND COMPLETENESS



2. *SOFTWARE DESIGN & IMPLEMENTATION*

Converting a system specification into an executable system through Design + Programming.

4 ACTIVITIES IN REQUIREMENTS ENGINEERING

Architectural Design

OVERALL STRUCTURE, PRINCIPAL COMPONENTS, THEIR RELATIONSHIPS AND DISTRIBUTION

Interface Design

DEFINES INTERFACES BETWEEN SYSTEM COMPONENTS, MUST BE UNAMBIGUOUS

Component Design

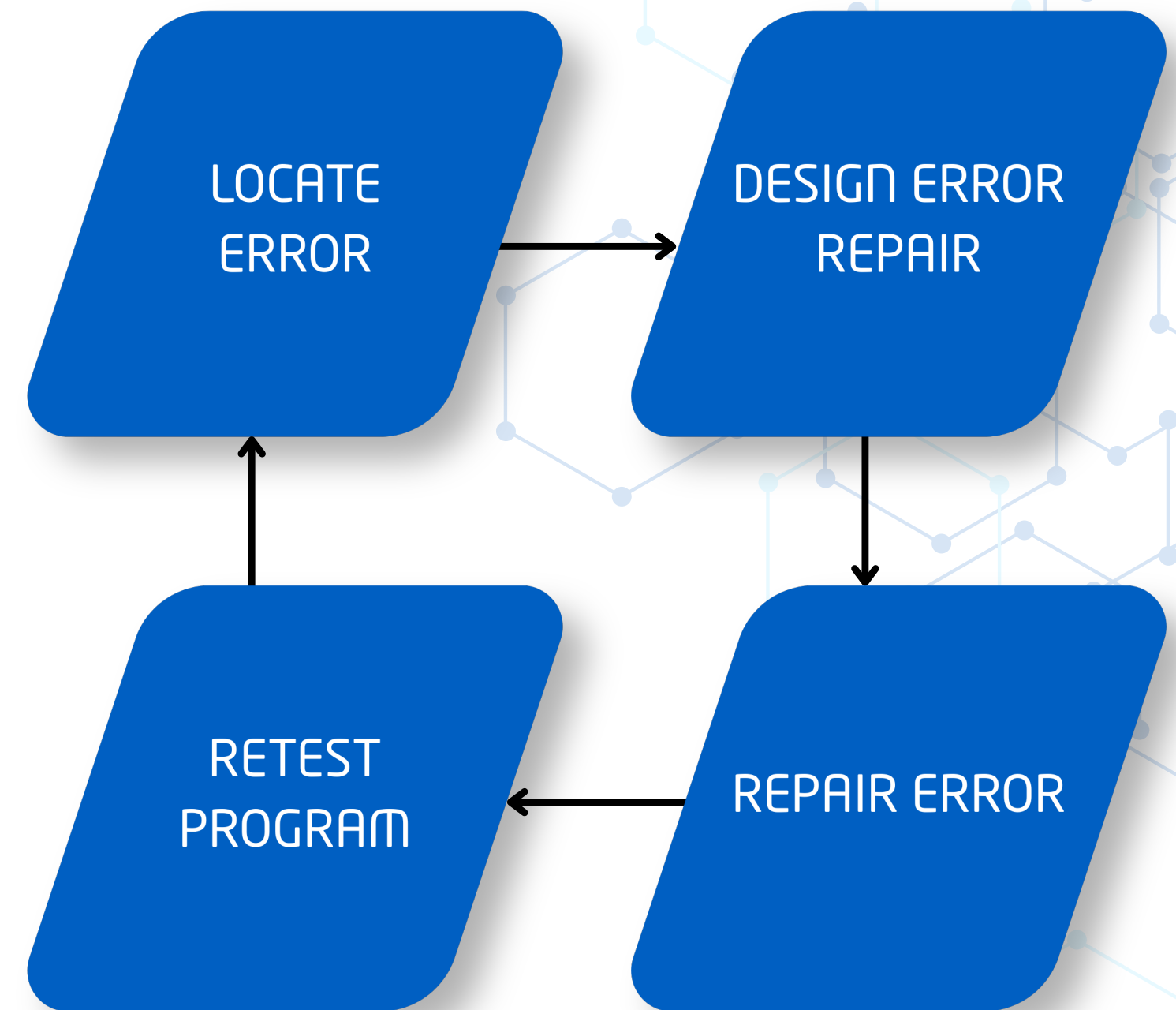
EACH COMPONENT'S INTERNAL OPERATION, MAY INCLUDE REUSABLE COMPONENT CHANGES

Database Design

DATA STRUCTURES AND HOW THEY ARE STORED, NEW OR REUSE EXISTING DATABASE

TESTING & DEBUGGING

- Programming is a personal activity, no single universal process
- Some start with familiar components, others tackle hard parts first
- Programmers test their own code during development
- **Testing:** establishes the EXISTENCE of defects
- **Debugging:** LOCATES and CORRECTS those defects
- Debugging is part of both Development AND Testing stages



3. SOFTWARE VALIDATION (V&V)

Showing that the system CONFORMS to its specification AND meets CUSTOMER expectations.

3 STAGES OF THE TESTING PROCESS

1. Component / Unit Testing

- Each component tested independently
- No interaction with other system components
- Tests functions, classes, or modules

2. System Testing

- All components integrated together
- Finds unexpected interaction errors
- Tests functional & non-functional requirements

3. Acceptance Testing

- Tested with REAL customer data
- May reveal missing/wrong requirements
- Final stage before operational use

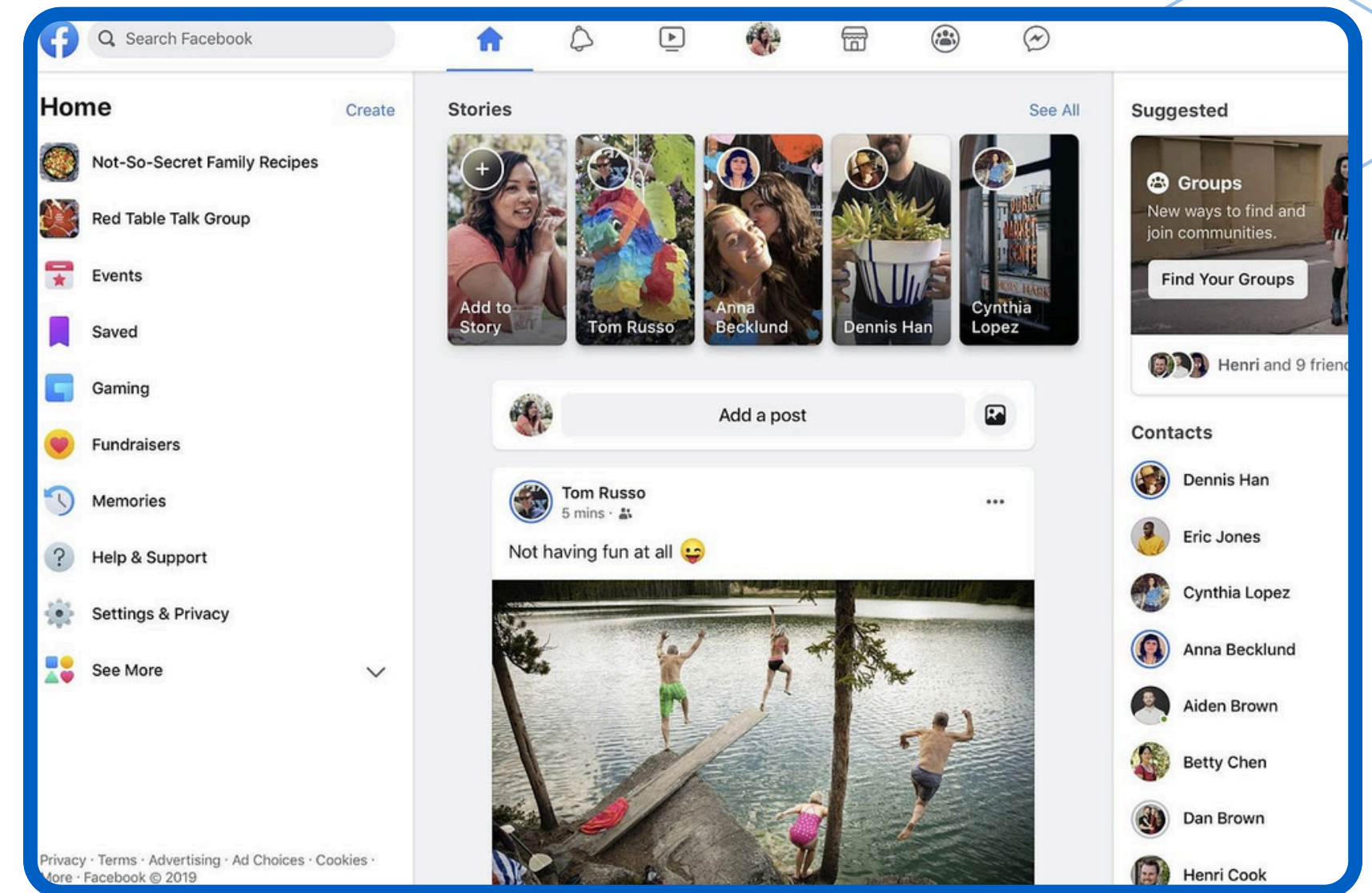
4. SOFTWARE EVOLUTION

updating and improving a software system over time, even after it has already been released and is being used.

EXAMPLE:



FACEBOOK (2014)



FACEBOOK (2026)

ADVANTAGES AND DISADVANTAGE OF PROCESS ACTIVITIES

Software Specification

Advantages:

- Reduces later design and implementation problems by defining clear system boundaries.
- Ensures business cost-effectiveness through early feasibility studies.

Disadvantages:

- Errors at this stage are critical and unavoidably lead to downstream failures.
- Interleaving of analysis and specification can make the timeline difficult to predict.

Software Design and Implementation

Advantages:

- Concurrent development is possible once interfaces are clearly defined.
- Design outputs provide a structured roadmap for programmers.

Disadvantages:

- Constant backtracking and design rework are inevitable as formality and detail are added.
- Design for complex systems can be highly expensive and time-consuming.

ADVANTAGE AND DISADVANTAGE OF PROCESS ACTIVITIES

Software Validation

Advantages:

- Shows the system conforms to its specification and meets customer expectations.
- Beta testing provides real-world feedback from potential customers.

Disadvantages:

- The majority of software costs are incurred during and after implementation.
- Identifying defects (testing) is separate from fixing them (debugging), adding complexity.

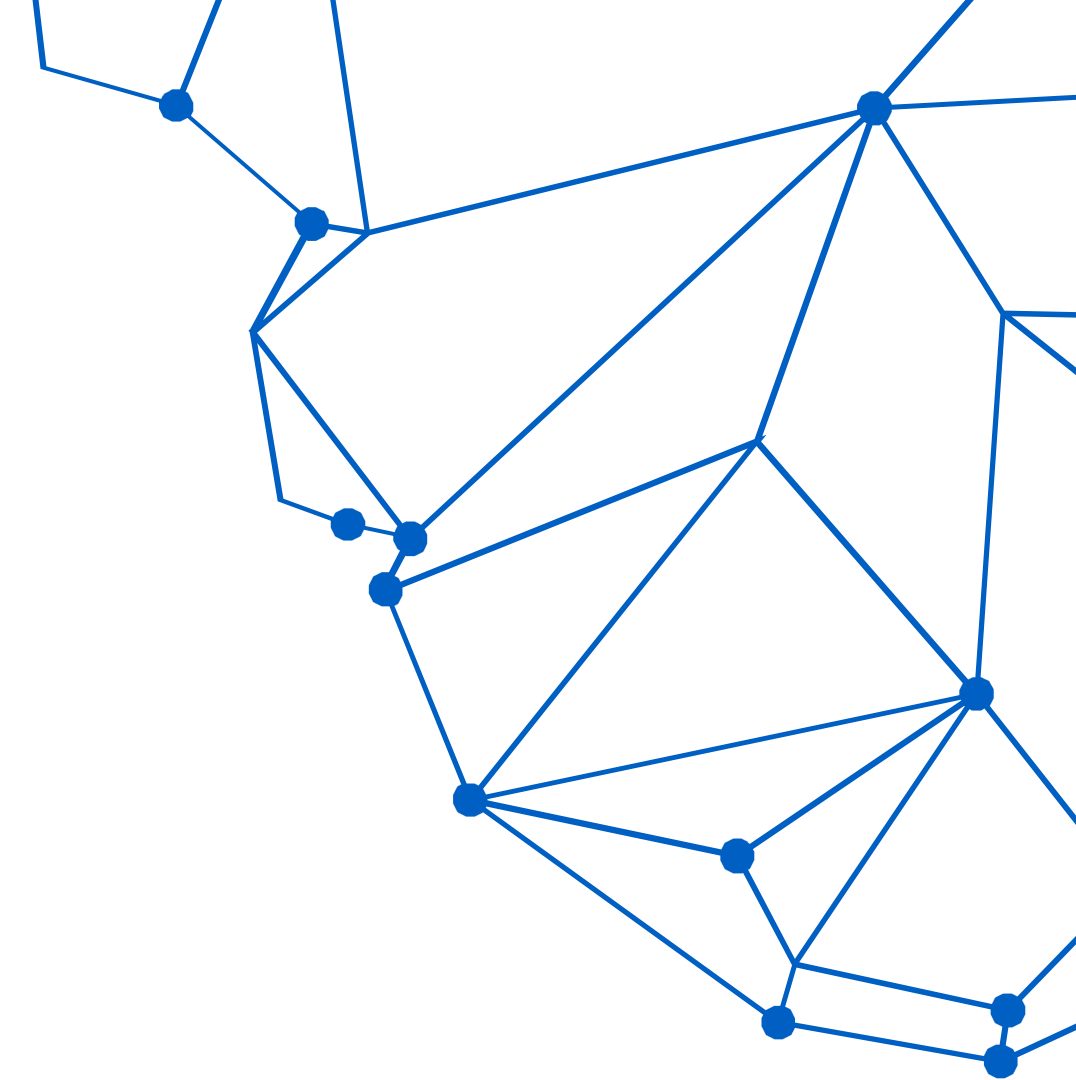
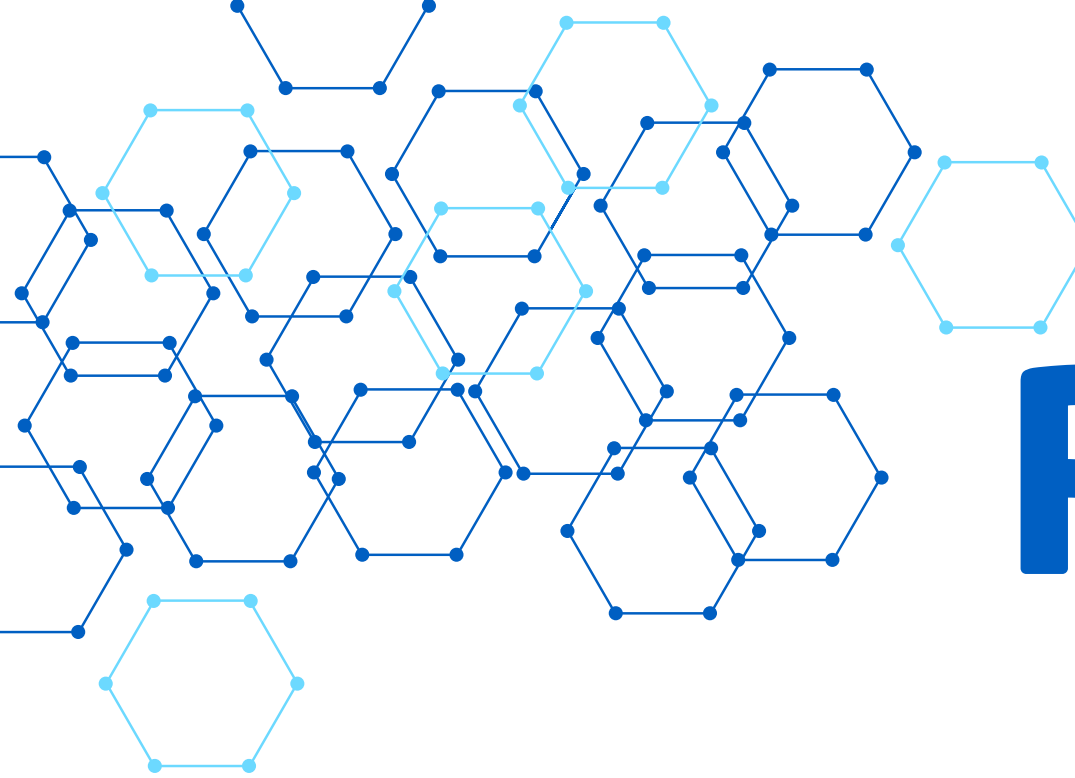
Software Evolution

Advantages:

- Software is much more flexible and cheaper to change than hardware.
- Continuous change allows the system to remain relevant to shifting customer needs.

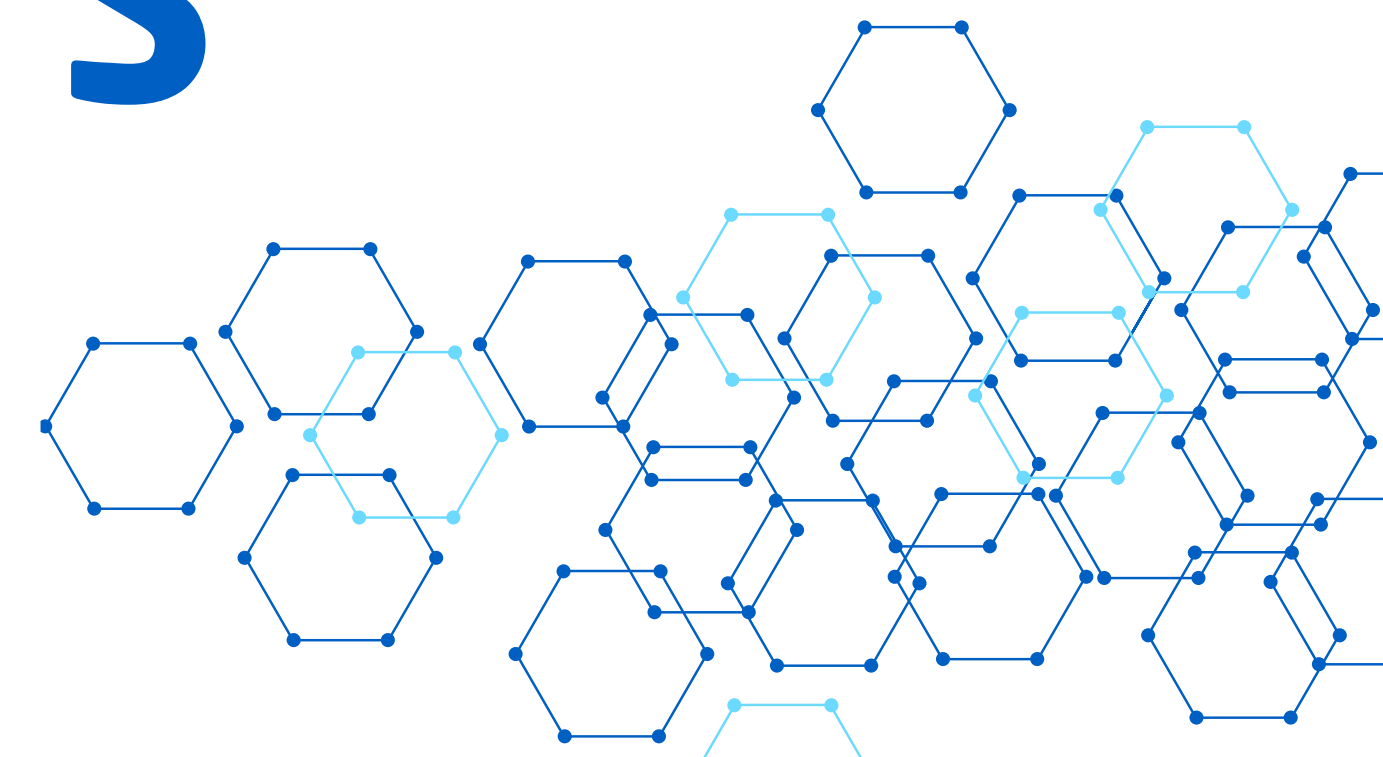
Disadvantages:

- Maintenance costs are often several times higher than initial development costs.
- Rework adds significant expense as previously completed tasks must be repeated.



RATIONAL UNIFIED PROCESS

REPORTER:
SHYR BELENZO



DEFINITION

The Rational Unified Process (RUP) is an iterative software development process framework created by Rational Software (now IBM). It combines proven software engineering best practices with iterative delivery and visual modeling using UML (Unified Modeling Language).



KEY CHARACTERISTICS:

1

ITERATIVE - BUILDS SOFTWARE IN REPEATED CYCLES (ITERATIONS)

2

USE-CASE DRIVEN - REQUIREMENTS ARE CAPTURED AS "USE CASES" (USER INTERACTIONS)

3

ARCHITECTURE-CENTRIC - FOCUSES ON A SOLID SOFTWARE ARCHITECTURE EARLY

4

CONFIGURABLE - CAN BE TAILORED TO FIT SMALL OR LARGE PROJECTS



THE FOUR PHASES OF RUP

Inception Phase

The Inception phase is the initial stage of the Rational Unified Process where the project's business case, scope, and feasibility are established. It involves identifying key stakeholders, defining the project's vision, estimating costs and schedules, and conducting an initial risk assessment.

Elaboration Phase

The Elaboration phase is the second stage of the Rational Unified Process where the system's architecture is defined, analyzed, and validated. This phase focuses on capturing a majority of the system requirements, mitigating high-risk technical and project risks, and building a prototype or proof-of-concept to test critical components.

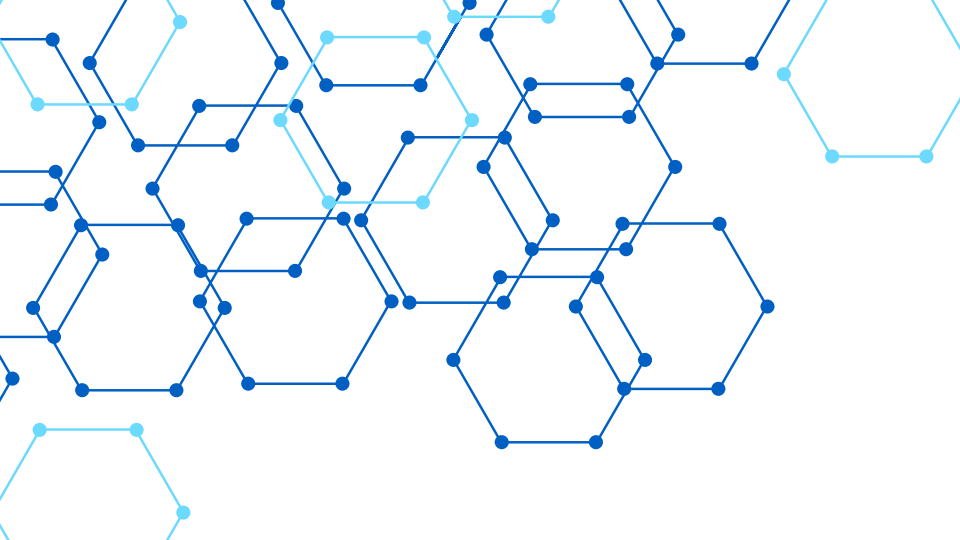
THE FOUR PHASES OF RUP

Construction Phase

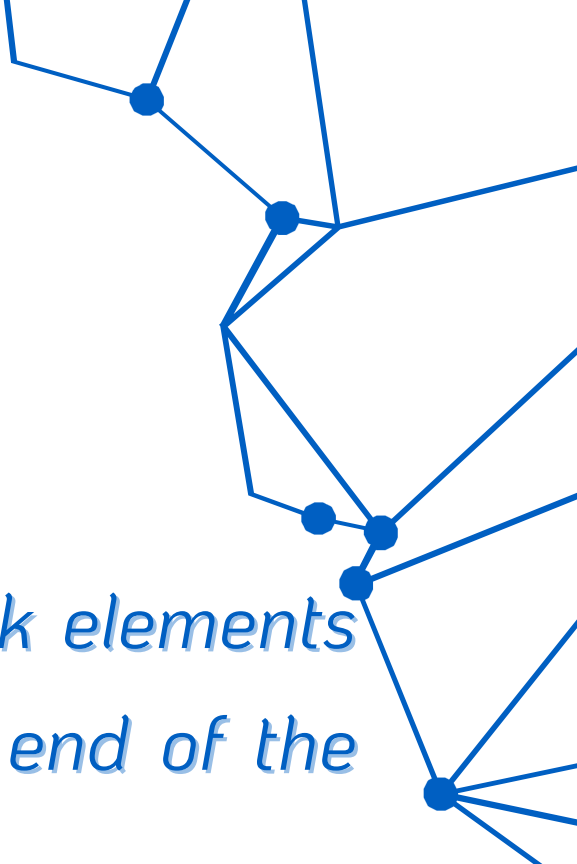
The Construction phase is the third and often longest stage of the Rational Unified Process where the software is built incrementally through multiple development iterations. Each iteration involves design, coding, testing, and integration, resulting in a progressively more complete and functional version of the product.

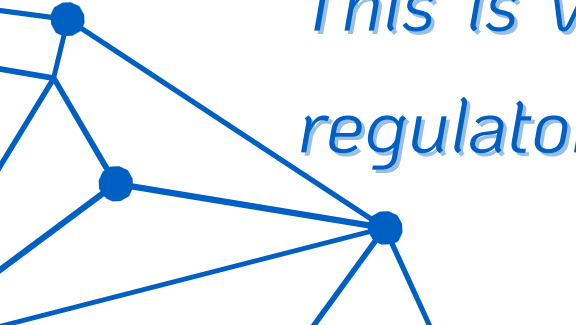
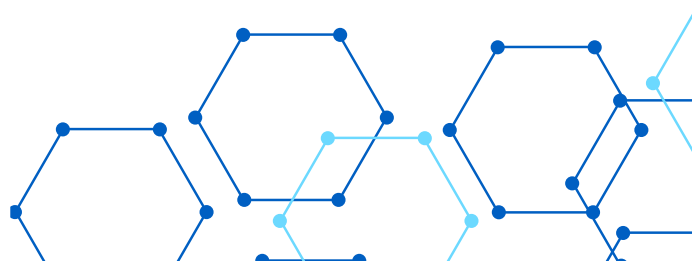
Transition Phase

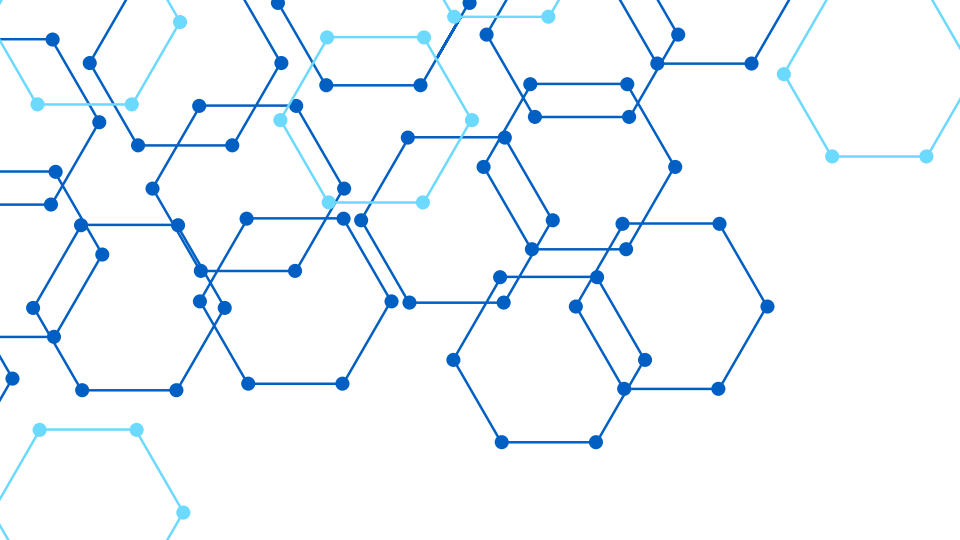
The Transition phase is the final stage of the Rational Unified Process where the software is delivered to end users in a production environment. This phase includes activities such as deploying the system, migrating data from legacy systems (if applicable), conducting user training, creating user documentation, and fixing defects discovered after deployment.



ADVANTAGES

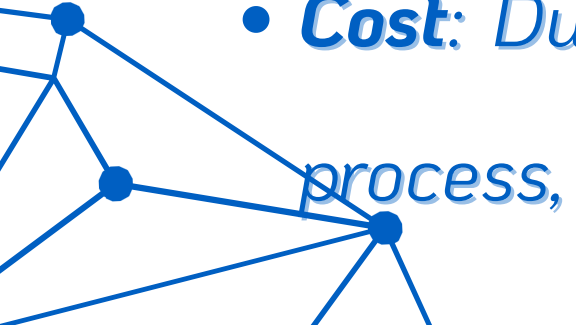
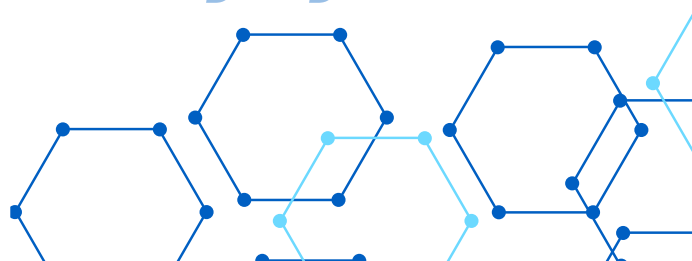


- **Risk Management:** One of RUP's biggest strengths is identifying and attacking high-risk elements early in the "Elaboration" phase. This prevents "nasty surprises" from appearing at the end of the project.
 - **Iterative Development:** Because it breaks development into smaller cycles, it allows for regular feedback and course correction, making it more flexible than the traditional Waterfall model.
 - **High Quality:** By integrating testing and quality verification into every iteration (rather than saving it for the end), the final product tends to be more robust and bug-free.
 - **Component Reuse:** The focus on architecture and modular components makes it easier to reuse parts of the code for future projects, saving time and money in the long run.
 - **Comprehensive Documentation:** RUP provides a clear "paper trail" for every stage of development. This is vital for large organizations that need to maintain systems over many years or meet strict regulatory requirements.
- 
- 



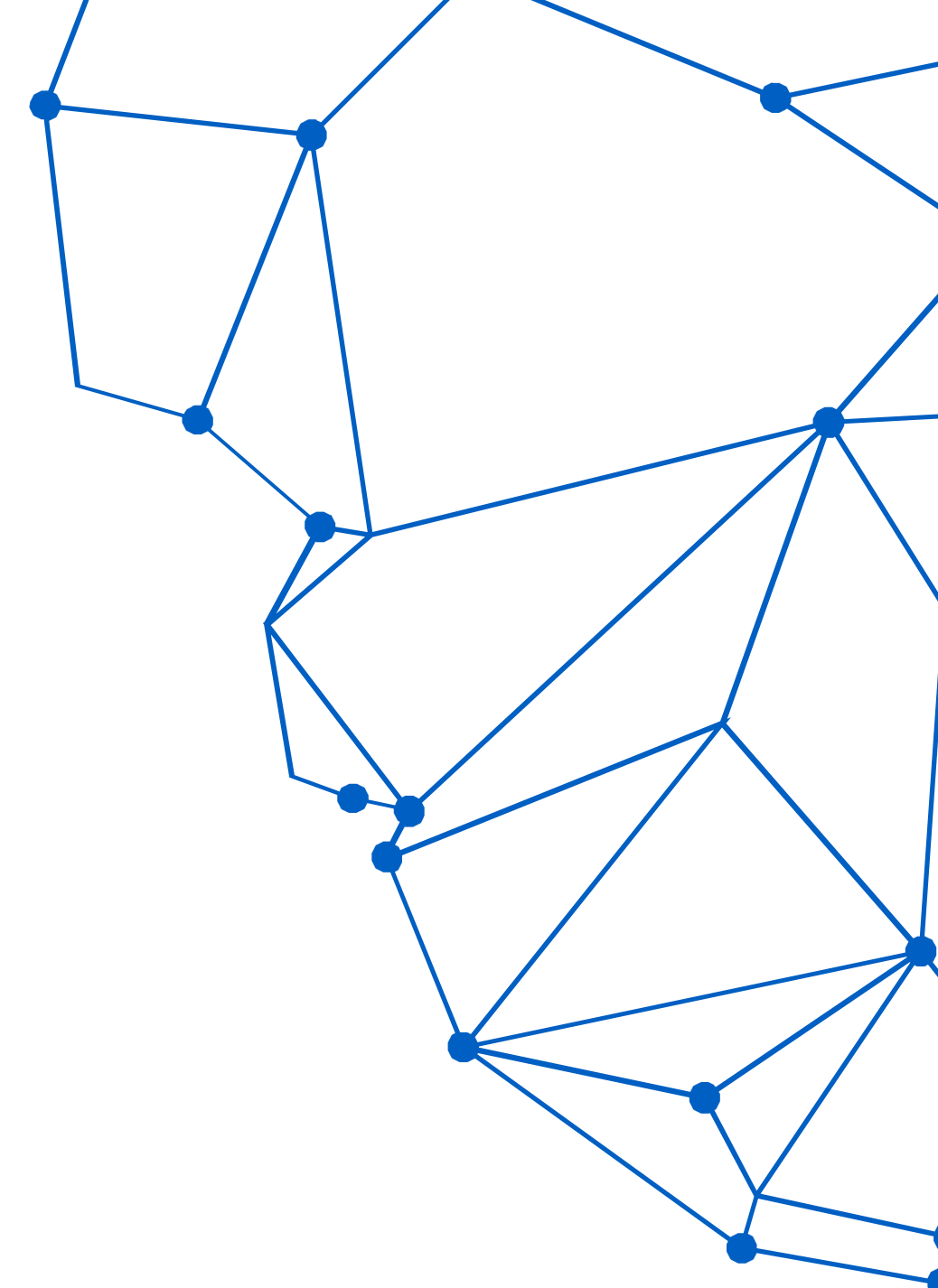
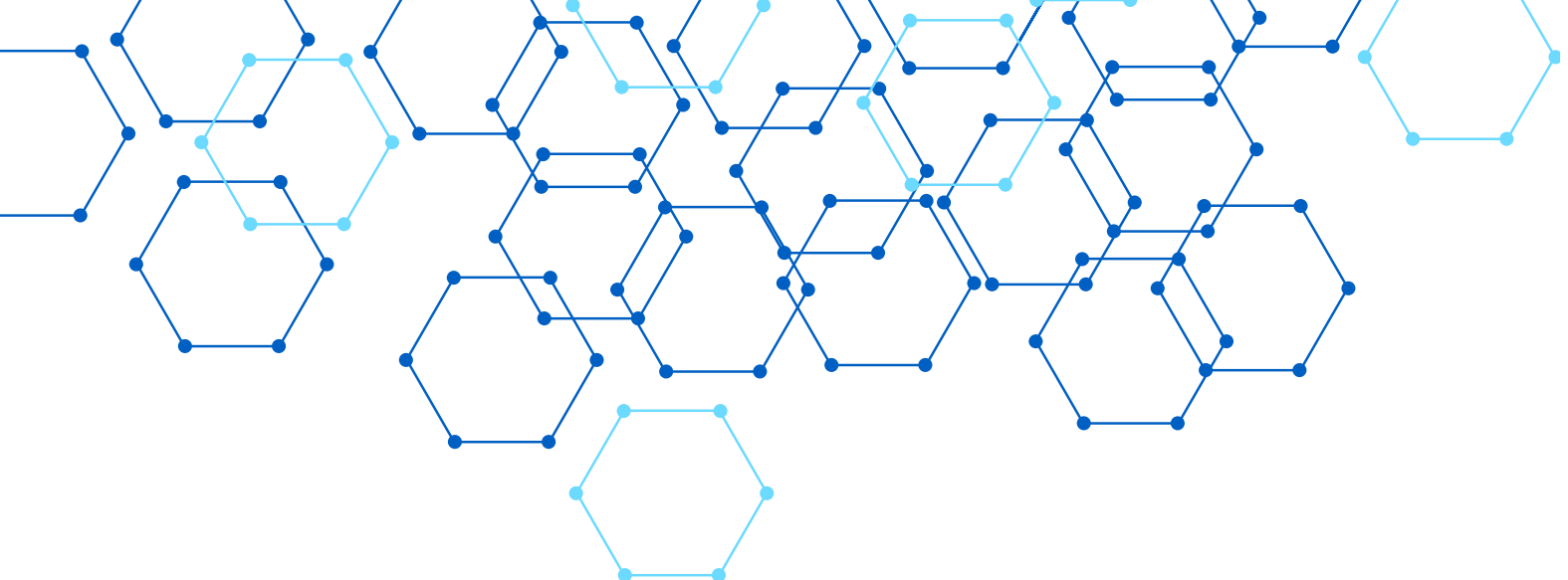
DISADVANTAGES



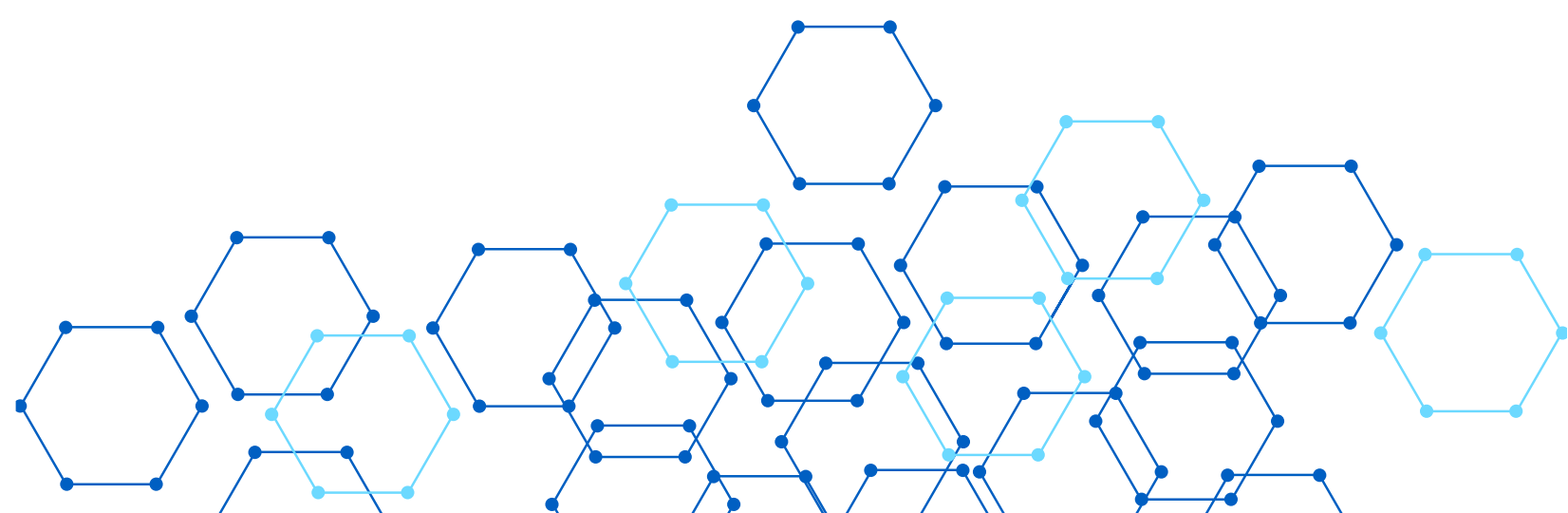
- **Complexity:** RUP is often described as "heavyweight." It requires a significant amount of expertise to set up and manage correctly, which can be overwhelming for smaller teams.
 - **Resource Intensive:** Because it demands detailed documentation and specialized roles (like separate architects and process engineers), it can consume a lot of time and man-hours.
 - **Not Beginner-Friendly:** To get the most out of RUP, team members need to be well-versed in UML (Unified Modeling Language) and the specific RUP methodology. The learning curve is quite steep.
 - **"Analysis Paralysis":** There is a risk that the team spends too much time on the Inception and Elaboration phases (planning and modeling) and not enough time actually writing the code.
 - **Cost:** Due to the need for specific tools (like IBM Rational tools) and the overhead of managing the process, RUP can be significantly more expensive than "leaner" methods like Agile.
- 
- 

CONCLUSION

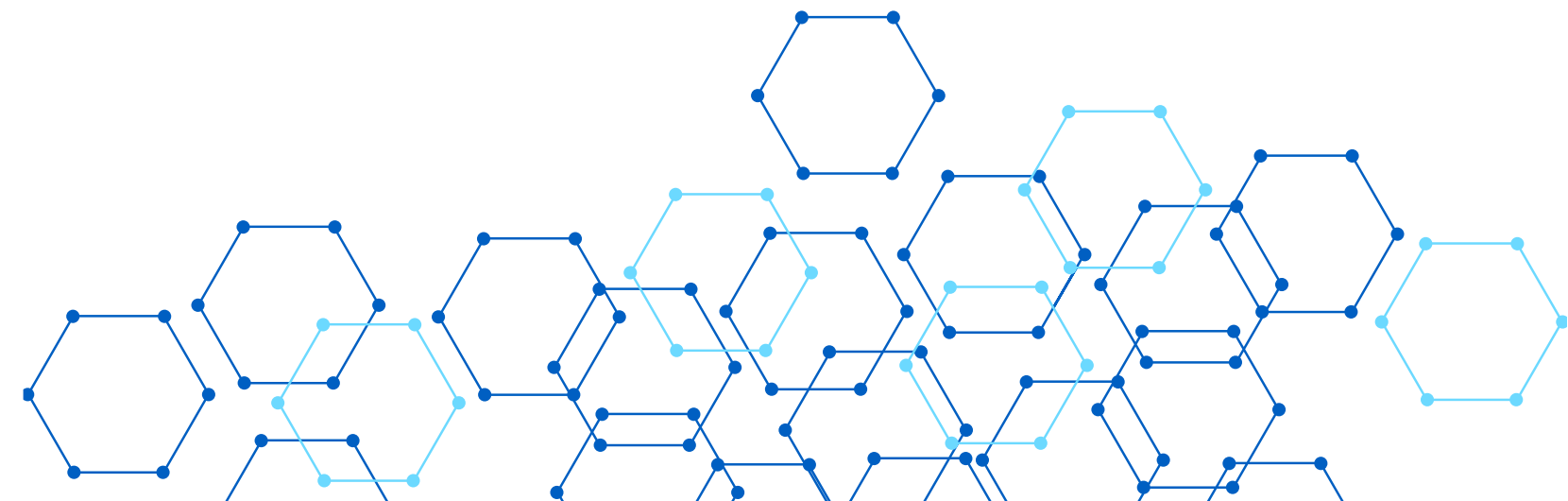
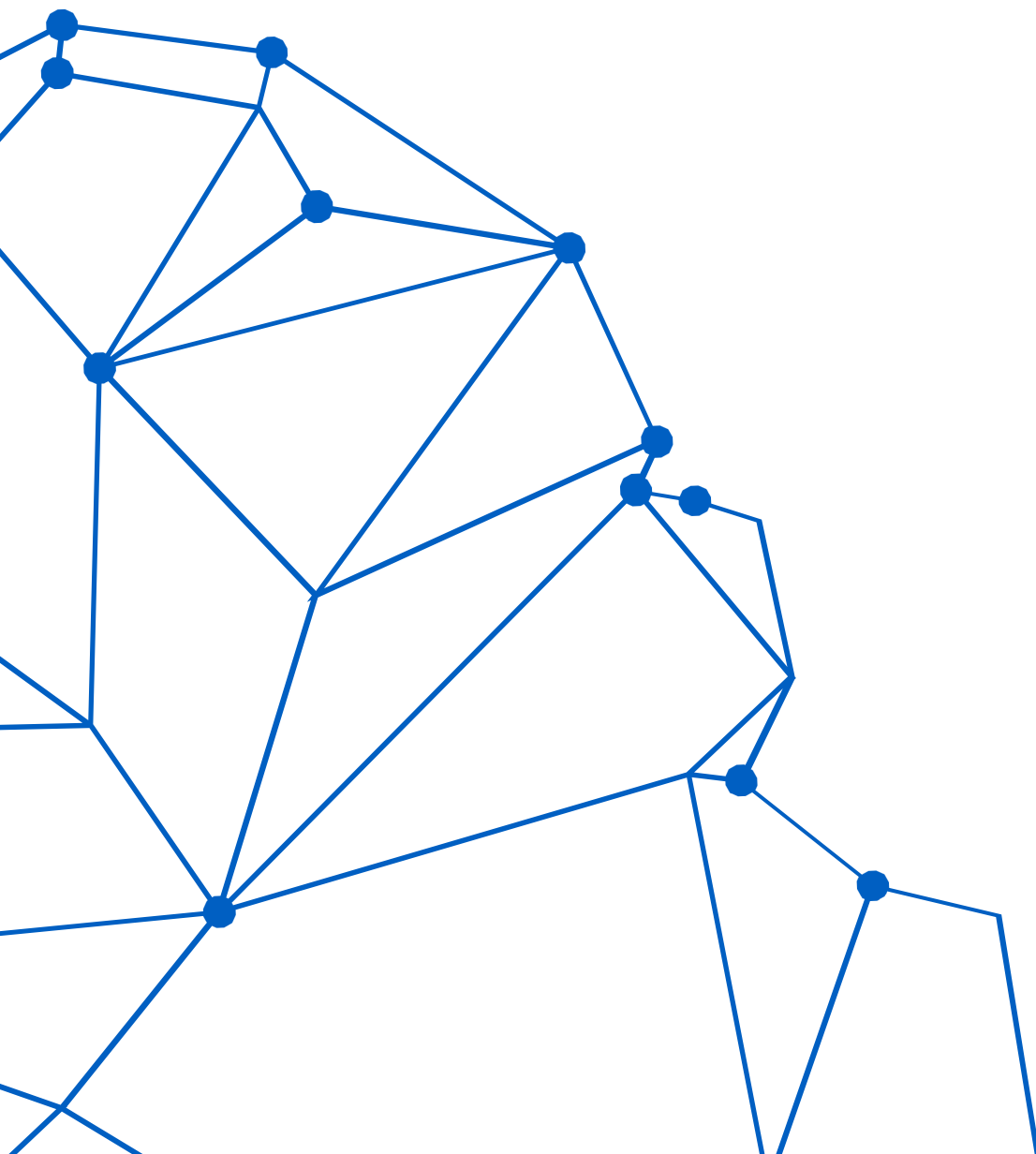
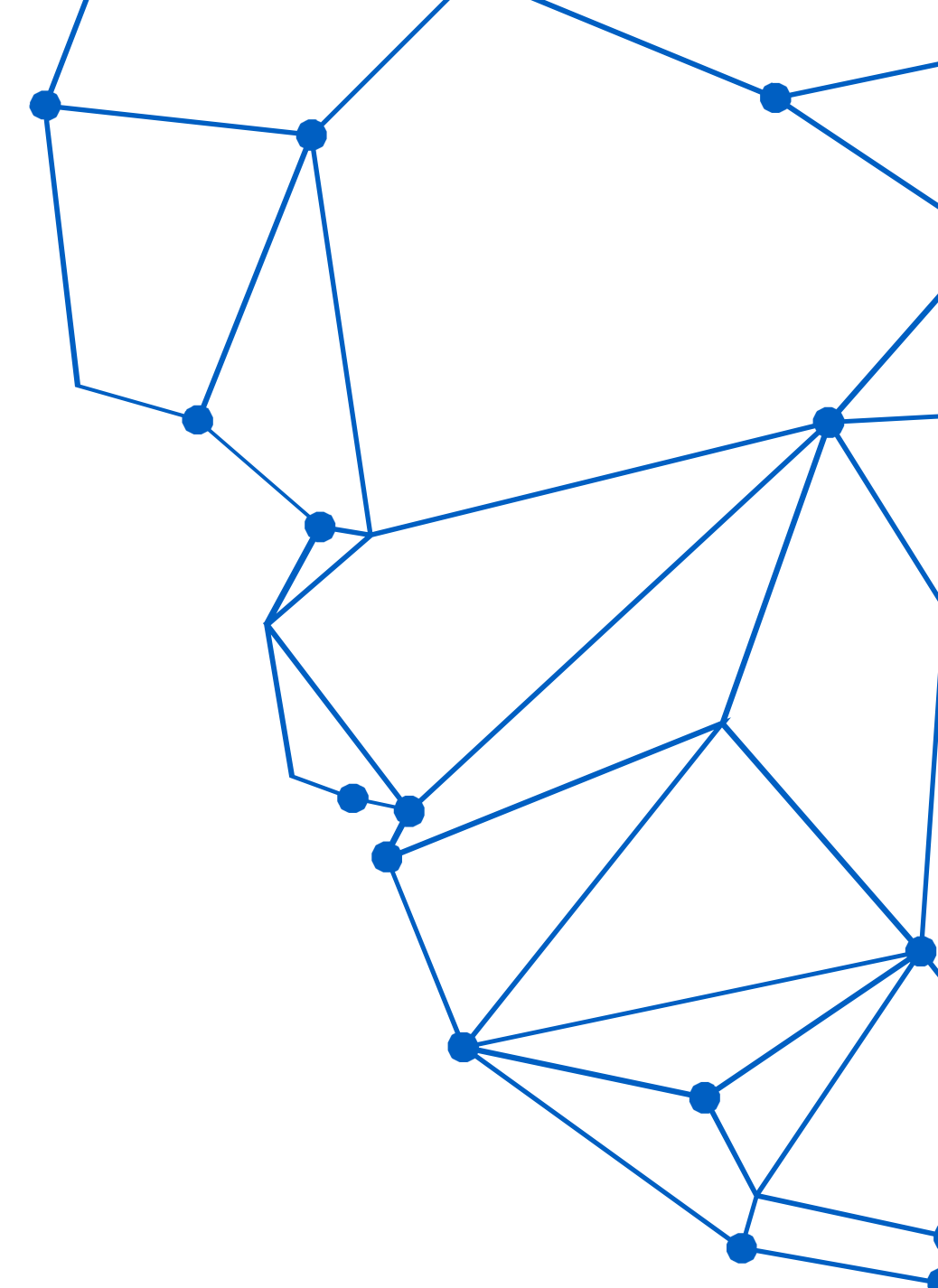
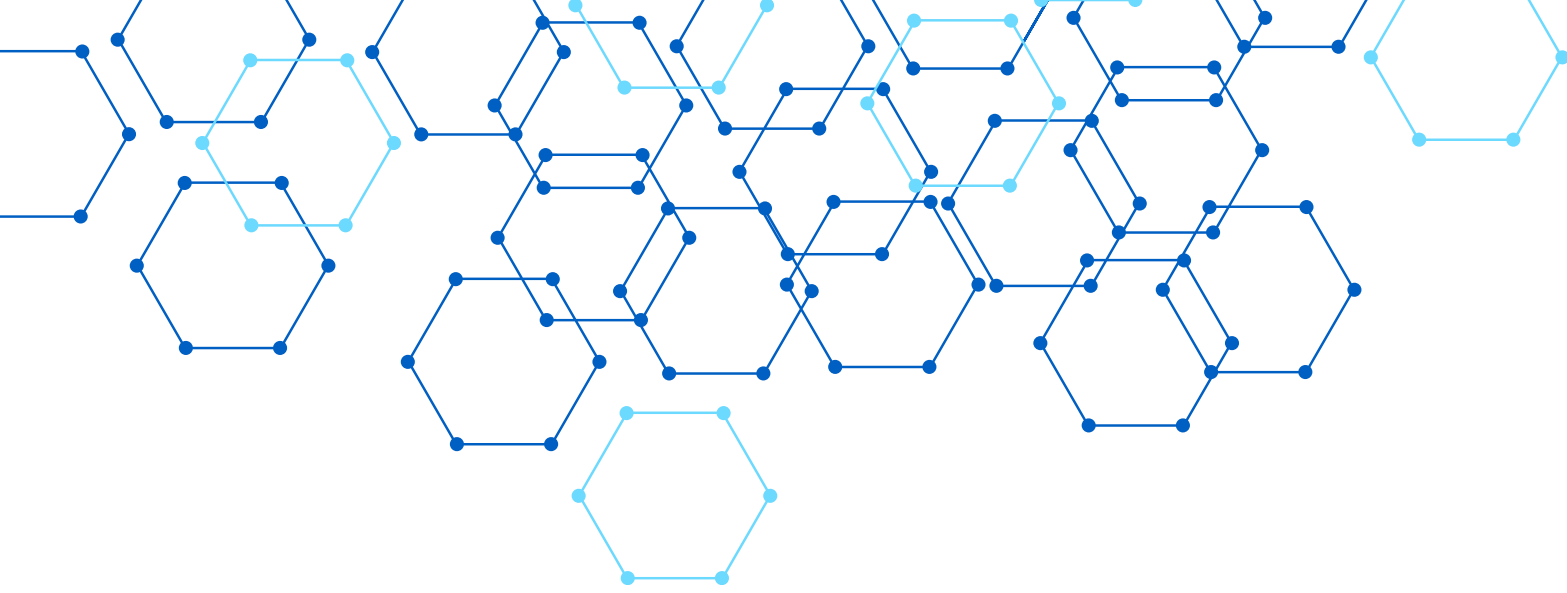
- In conclusion, the Rational Unified Process (RUP) represents a sophisticated, "middle-ground" approach to software development. It successfully combines the disciplined structure of traditional models with the adaptive, iterative nature of modern engineering.
- **Best for Complexity:** RUP is most effective for large-scale, high-stakes projects where technical risks are high and architectural stability is a necessity.
- **Quality through Discipline:** By emphasizing visual modeling (UML) and continuous verification, it ensures a high-quality end product that is well-documented and easy to maintain.
- **A Strategic Choice:** While its "heavyweight" nature can be a burden for small teams or simple applications, its ability to tackle complex requirements makes it a powerful tool for enterprise-level development.
- Ultimately, RUP is not a "one-size-fits-all" solution, but rather a flexible framework. Its success depends on a team's ability to tailor the process—keeping the rigorous planning that ensures success while avoiding the administrative bloat that can slow down innovation.



THANK YOU

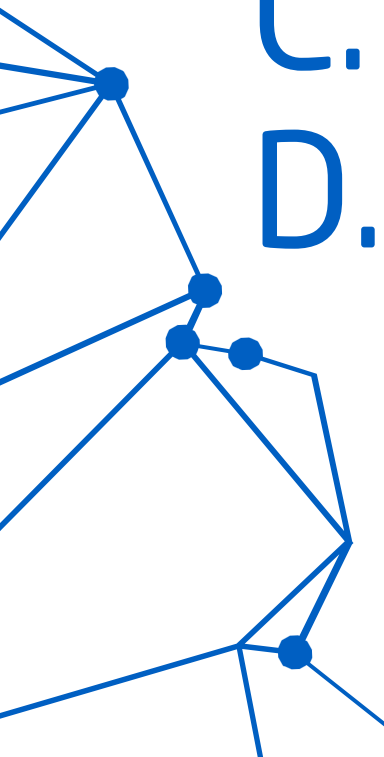
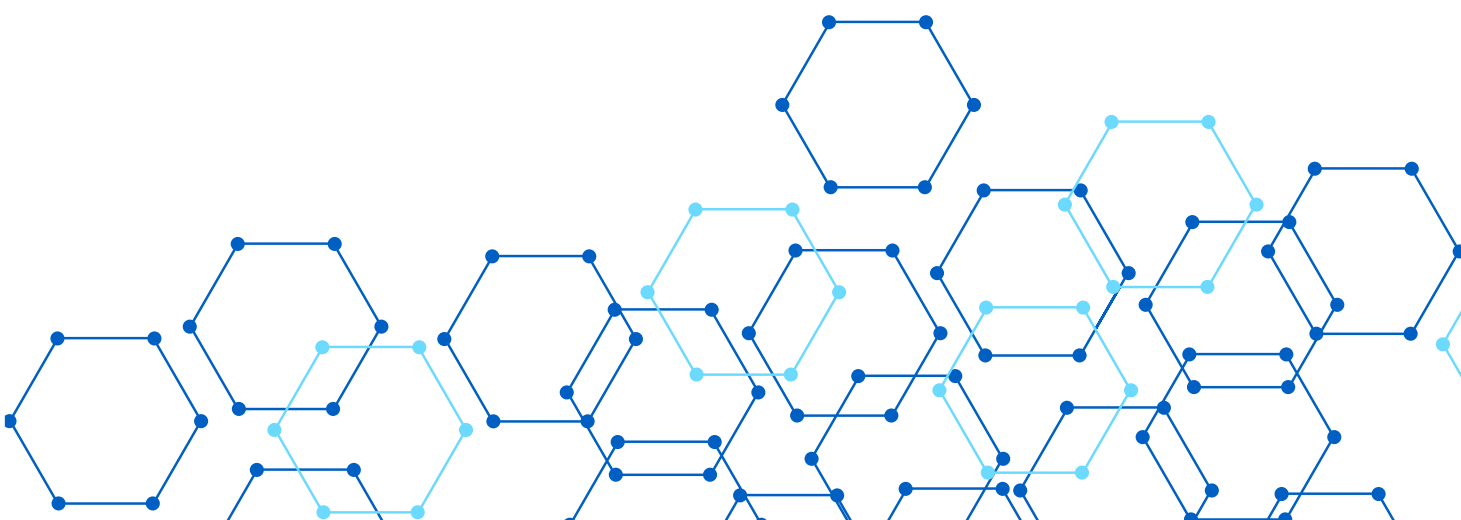


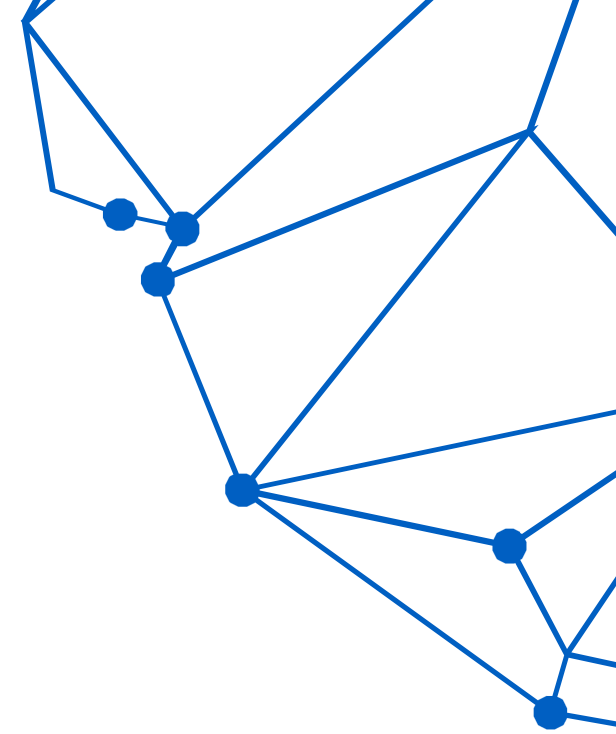
QUIZ



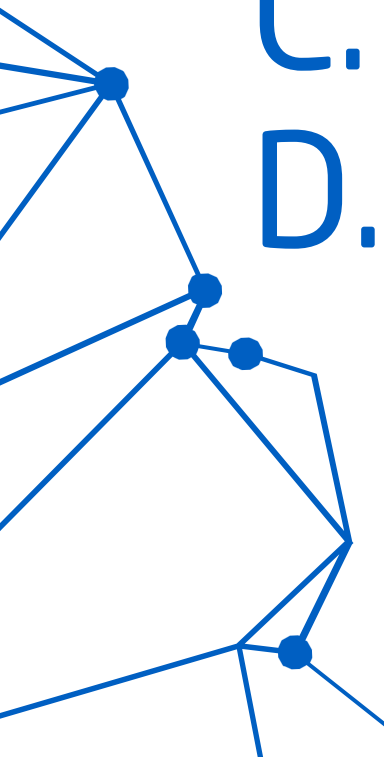
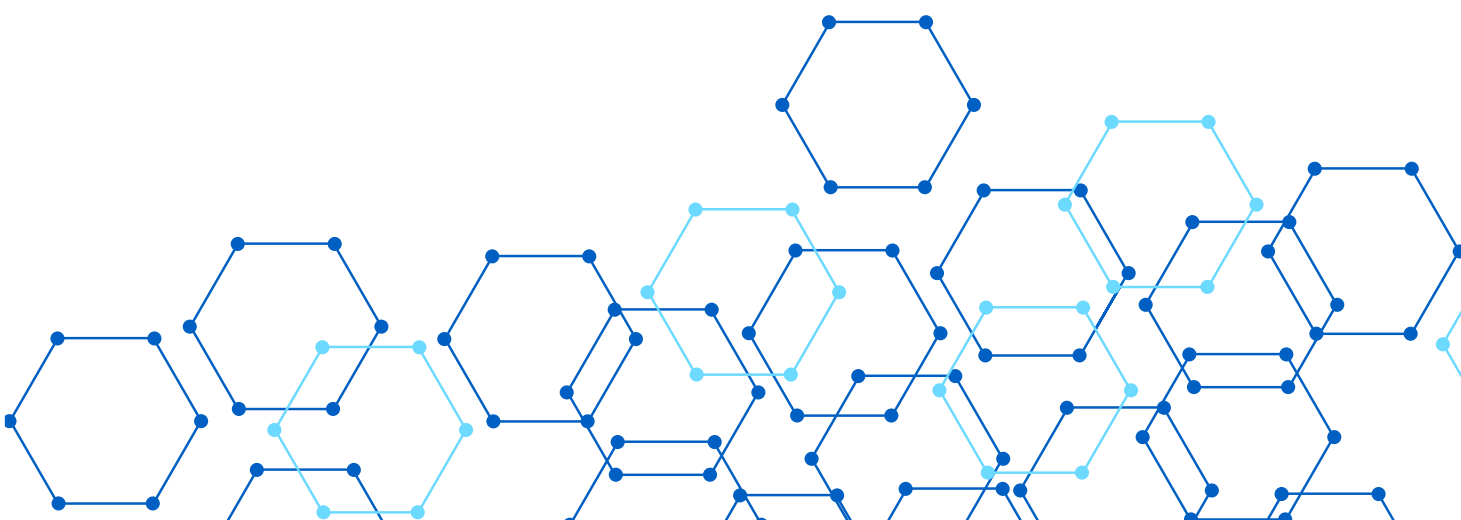


**1. WHICH TERM REFERS TO AN ABSTRACT,
SIMPLIFIED REPRESENTATION OF A
SOFTWARE DEVELOPMENT PROCESS?**

- A. Software Specification
 - B. Software Process Model
 - C. Feasibility Study
 - D. System Architecture
- 
- 

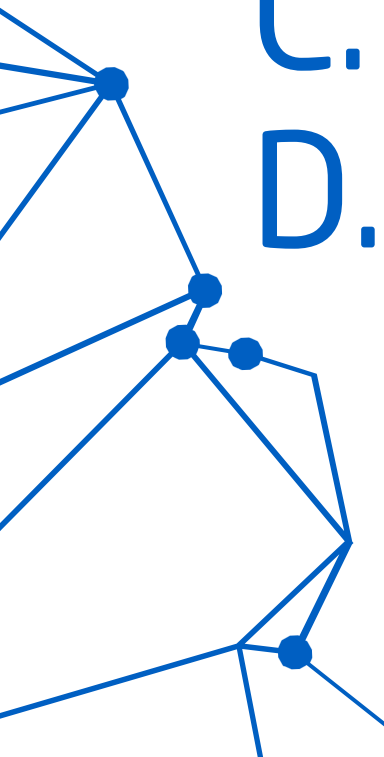


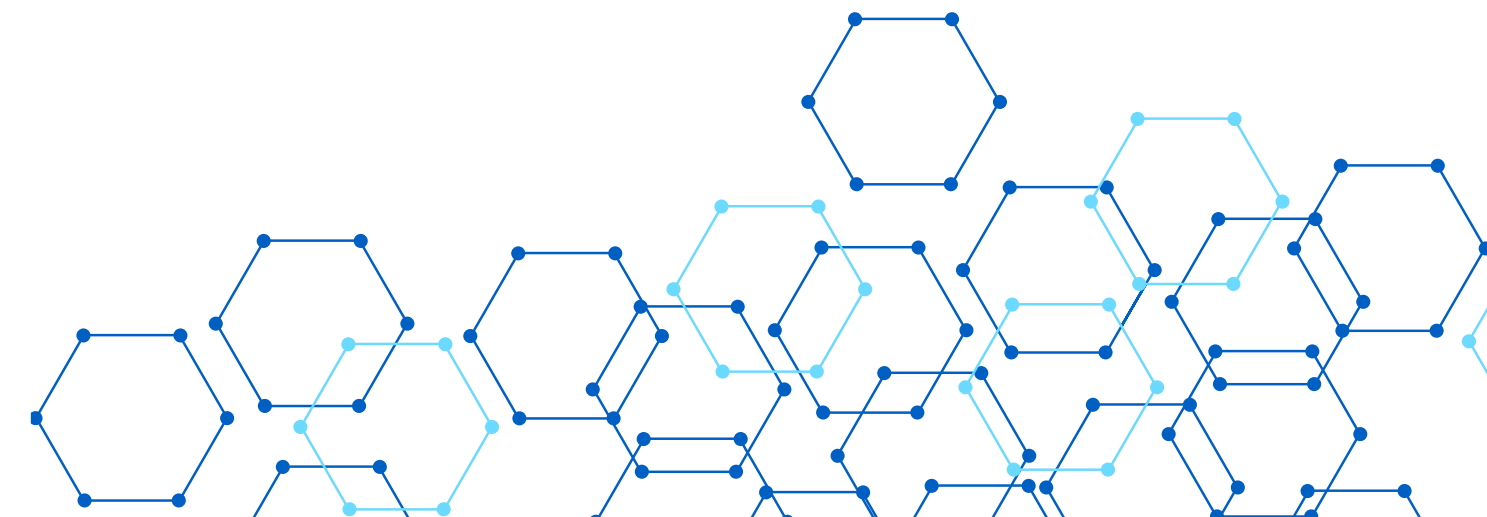
2. WHICH MODEL IS BEST SUITED FOR PROJECTS WHERE REQUIREMENTS ARE CLEAR AND FIXED?

- A. Agile
 - B. Spiral
 - C. Waterfall
 - D. Incremental
- 
- 



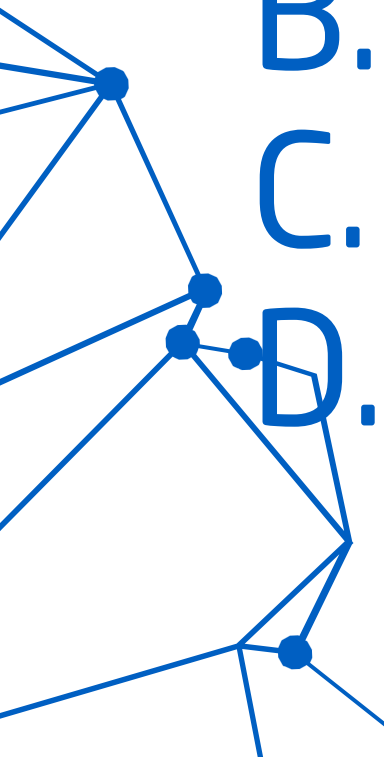
3.THE "4 BASIC PROCESS ACTIVITIES" ARE SPECIFICATION, DEVELOPMENT, VALIDATION, AND...

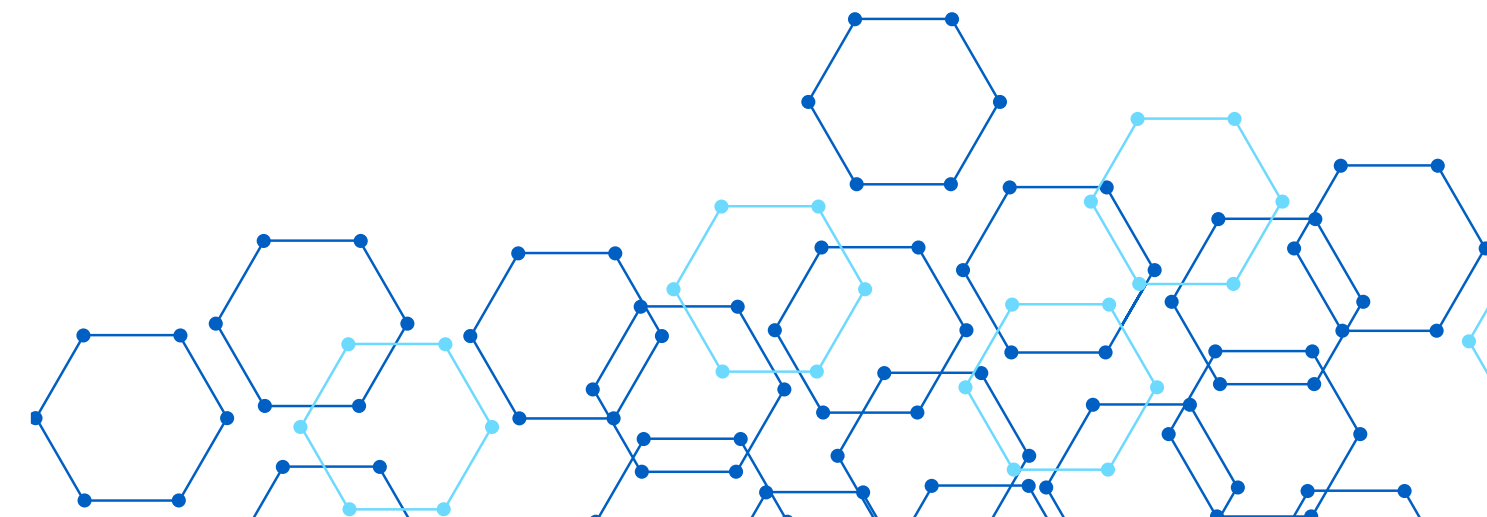
- A. Documentation
 - B. Evolution
 - C. Designing
 - D. Researching
- 





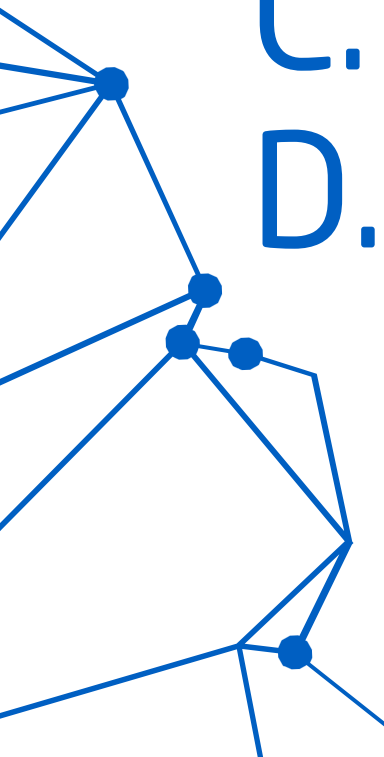
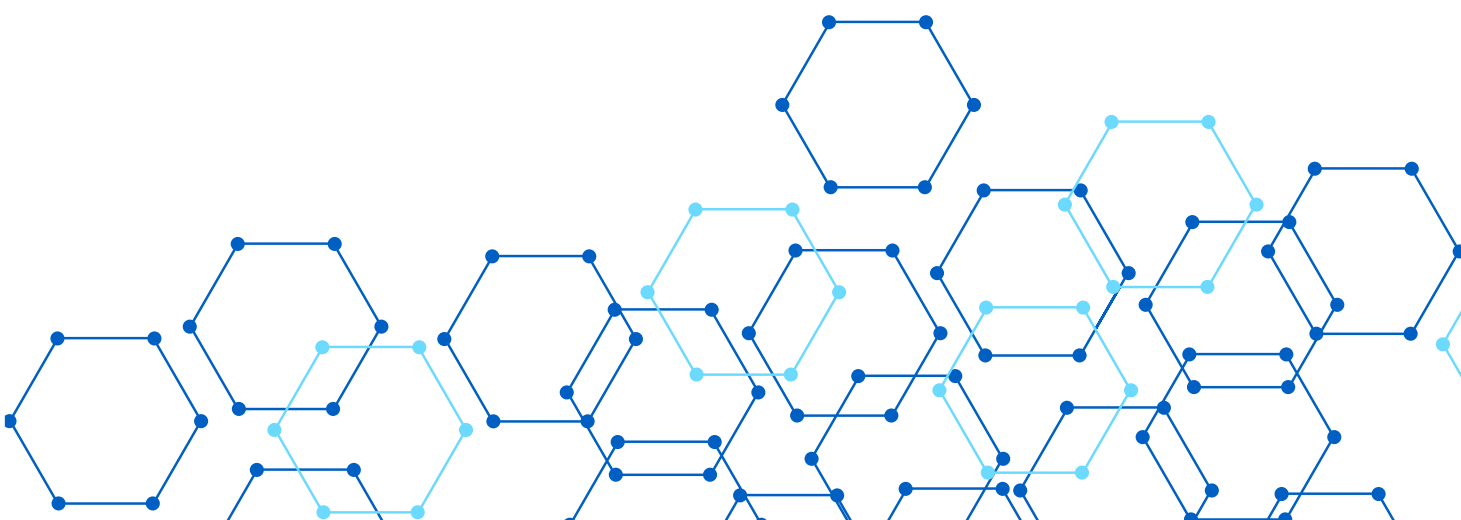
4. IN THE RATIONAL UNIFIED PROCESS (RUP), DURING WHICH PHASE IS THE PROJECT'S BUSINESS CASE AND FEASIBILITY ESTABLISHED?

- 
- A. Inception
 - B. Elaboration
 - C. Construction
 - D. Transition



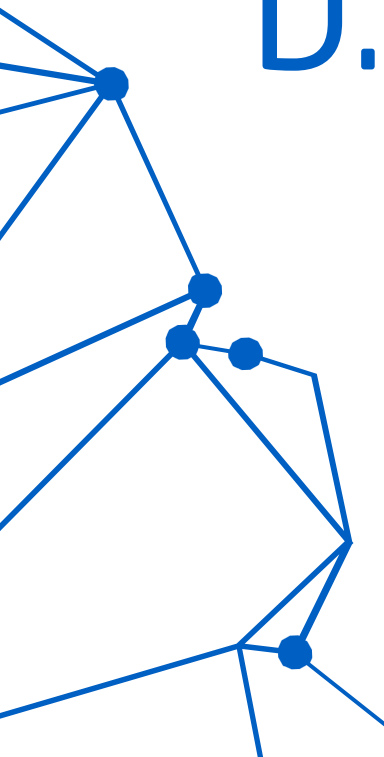
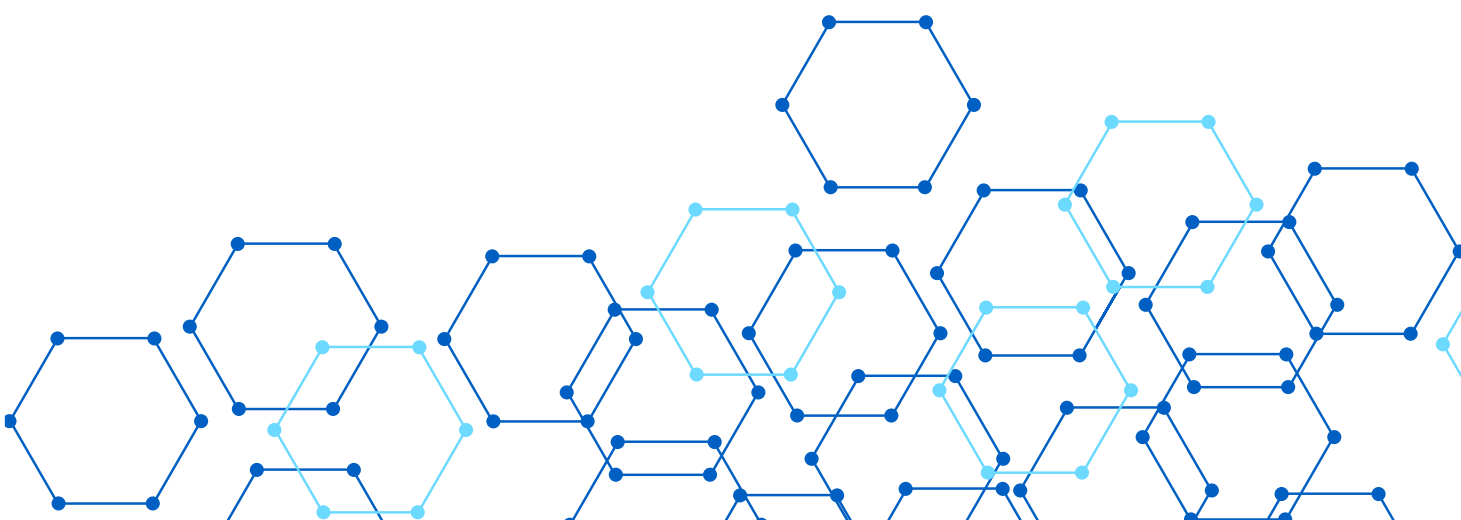


5. WHICH ACTIVITY INVOLVES DETERMINING IF A PROJECT IS POSSIBLE AND COST-EFFECTIVE?

- A. Requirements Validation
 - B. Architectural Design
 - C. Feasibility Study
 - D. Debugging
- 
- 

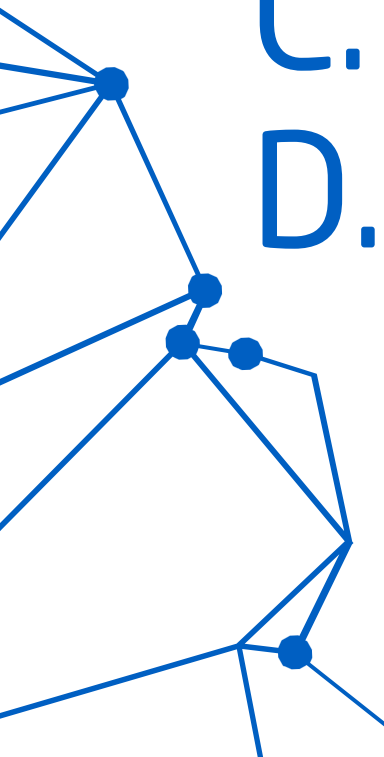
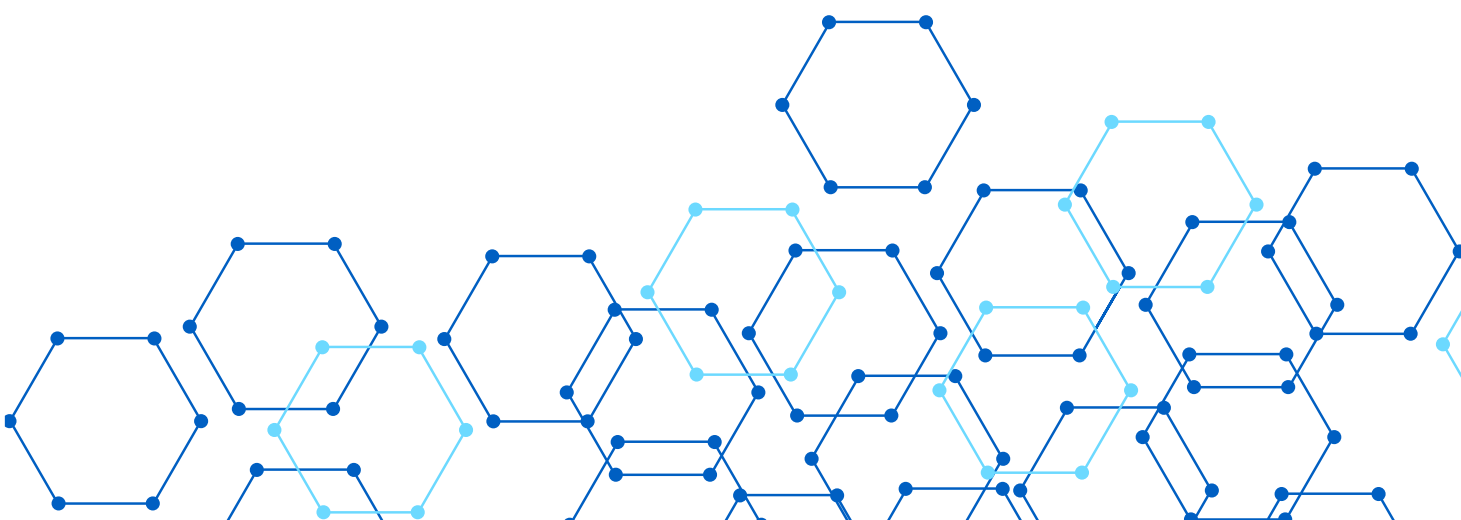


6. WHAT IS THE PRIMARY FOCUS OF THE SPIRAL MODEL?

- A. Linear progression through phases
 - B. Minimal documentation
 - C. Risk analysis and assessment
 - D. One-time delivery
- 
- 

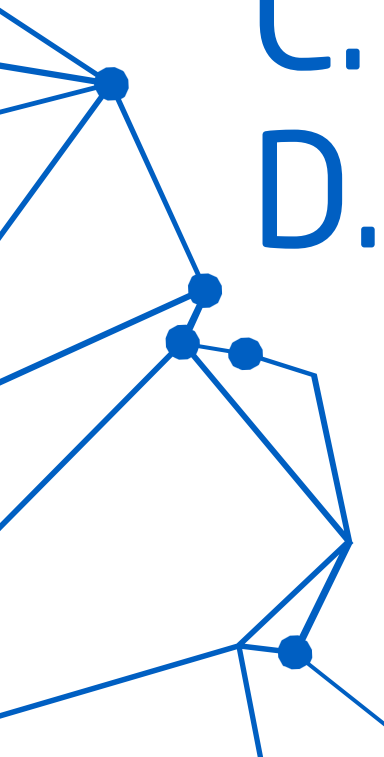
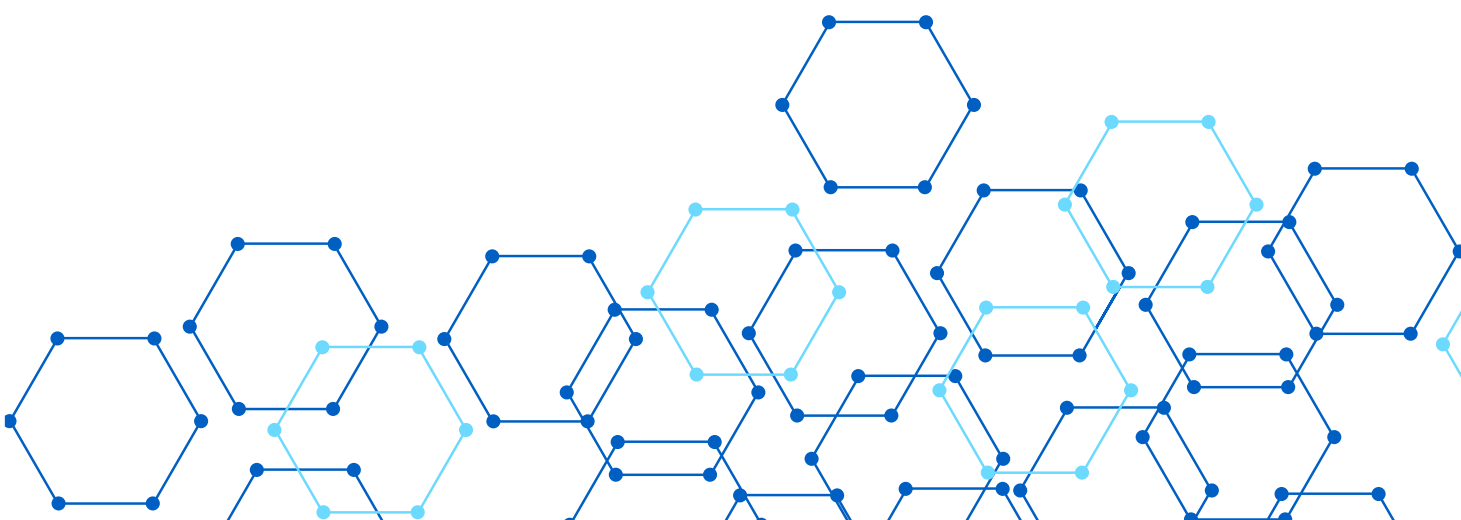


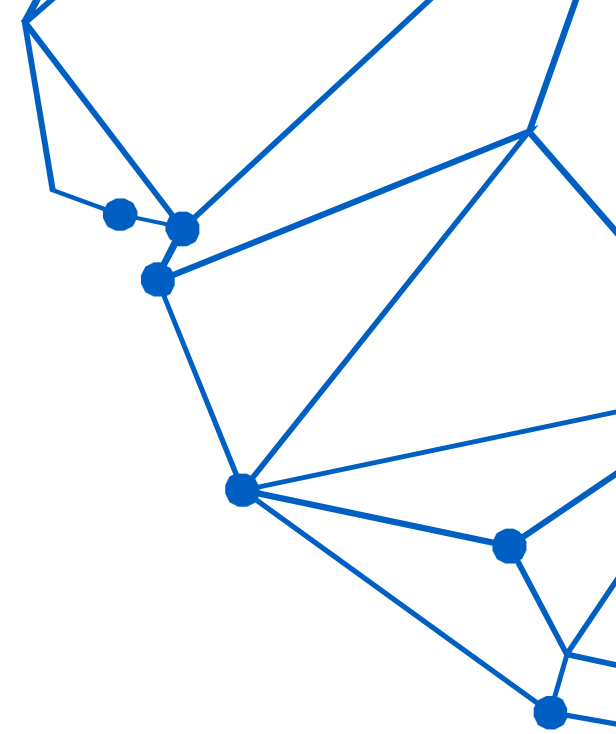
7. WHICH STAGE OF TESTING INVOLVES USING REAL CUSTOMER DATA BEFORE THE FINAL OPERATIONAL USE?

- A. Component Testing
 - B. System Testing
 - C. Acceptance Testing
 - D. Unit Testing
- 
- 

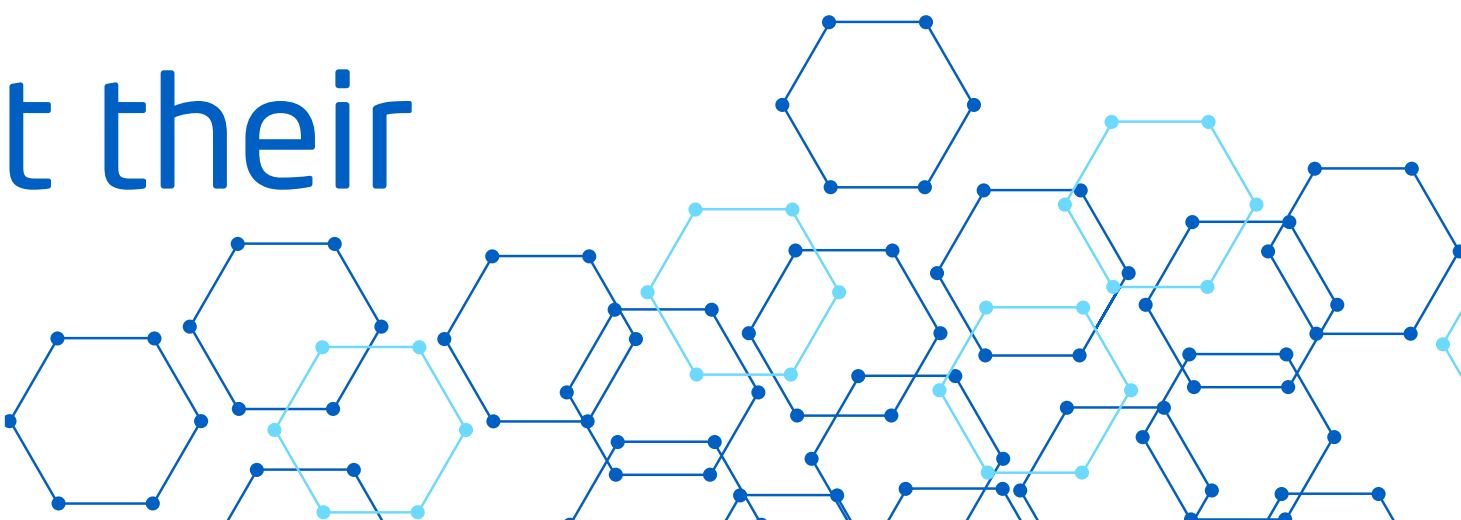


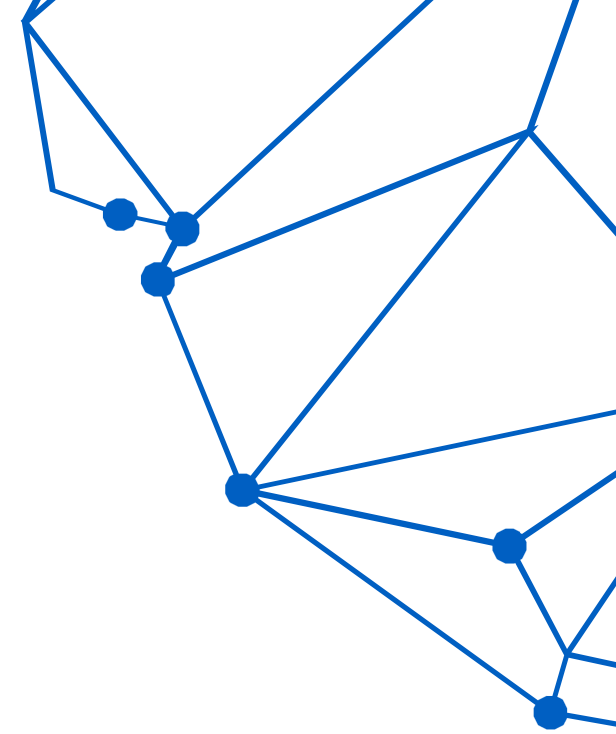
8. WHICH PHASE OF RUP IS TYPICALLY THE LONGEST AND INVOLVES INCREMENTAL CODING AND TESTING?

- A. Inception
 - B. Elaboration
 - C. Construction
 - D. Transition
- 
- 

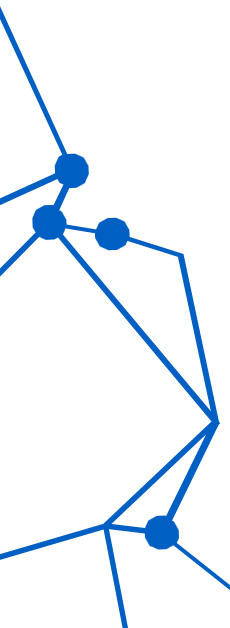


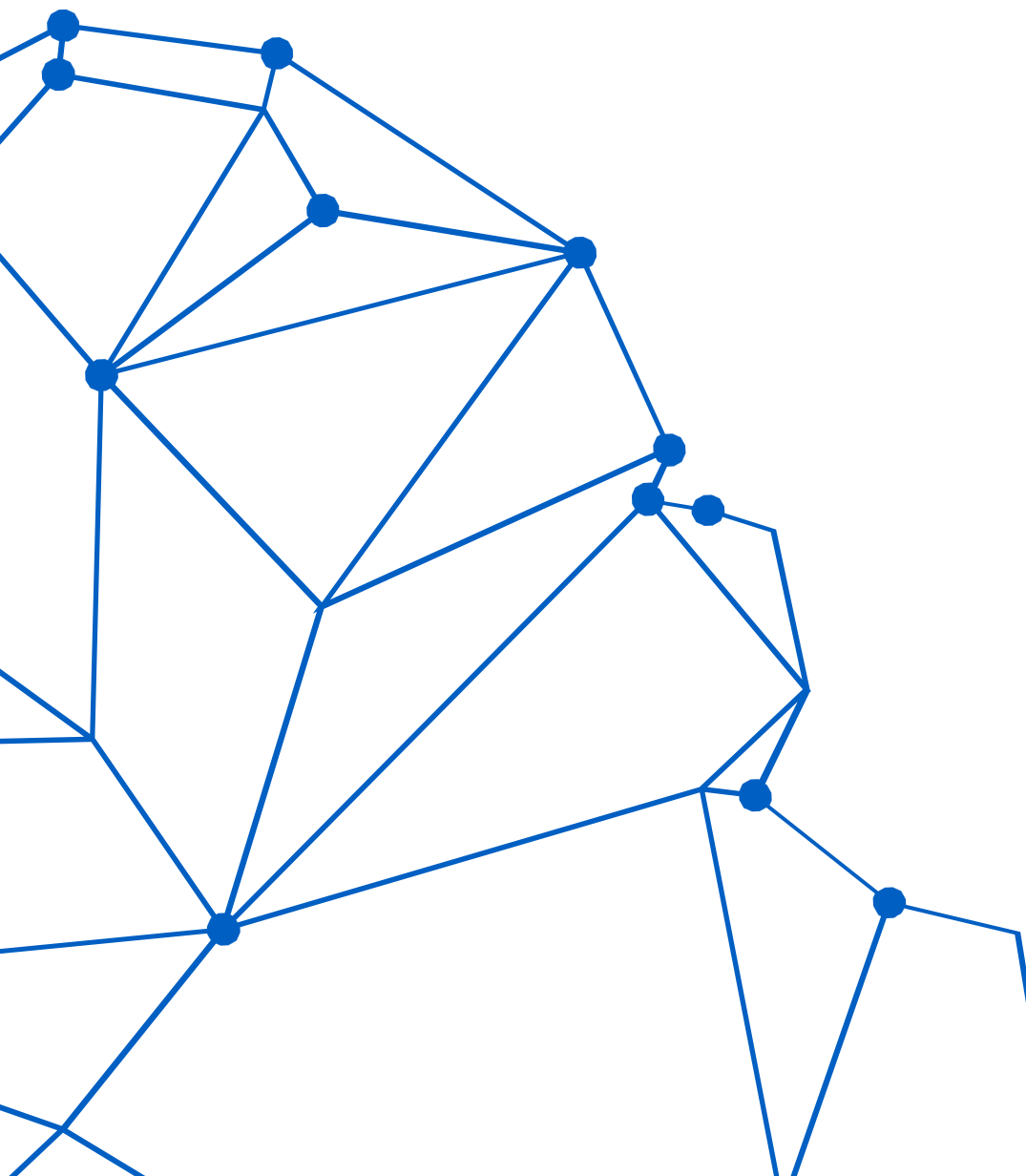
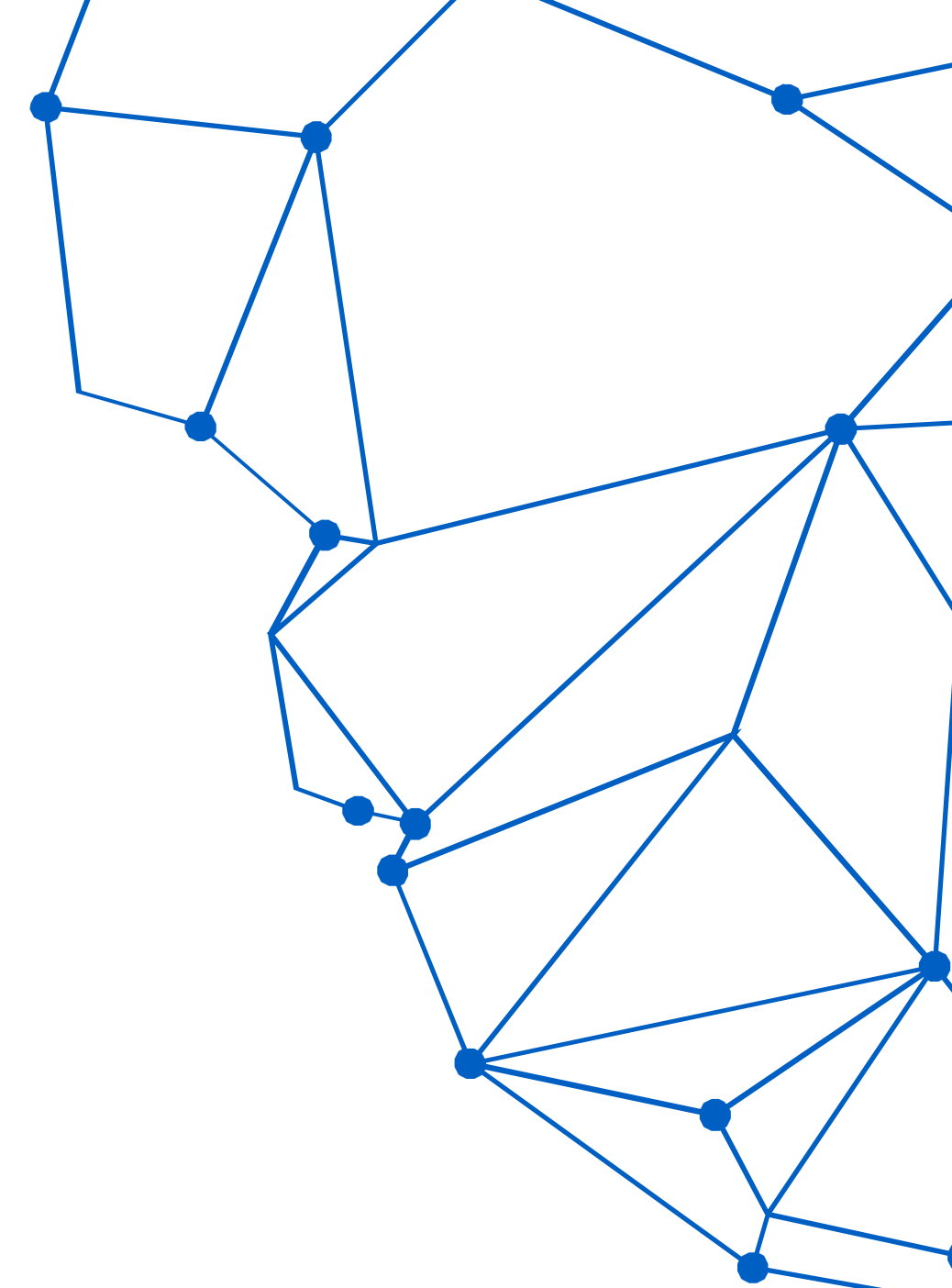
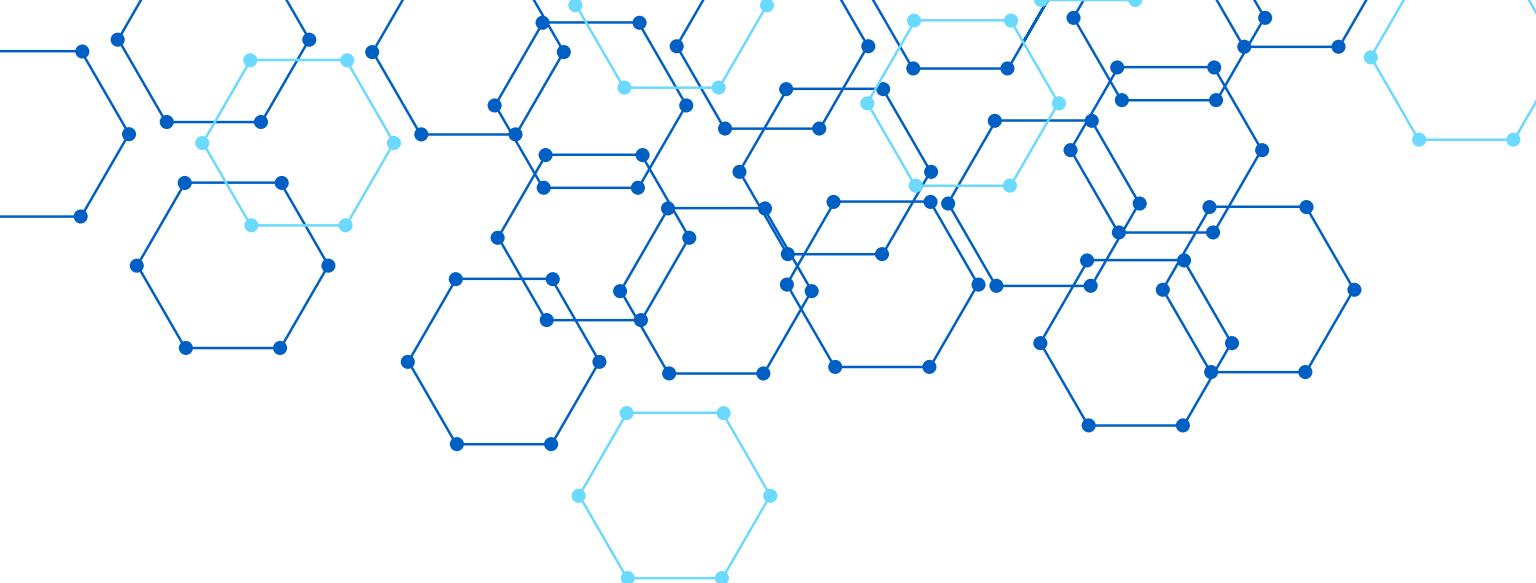
9.A GOOD PROCESS MODEL SHOULD BE "REPEATABLE." THIS MEANS:

- A. It uses the same code for every project.
 - B. It can be applied across different projects for consistent results.
 - C. It requires the team to repeat the design phase twice.
 - D. It allows customers to repeat their requirements indefinitely.
- 

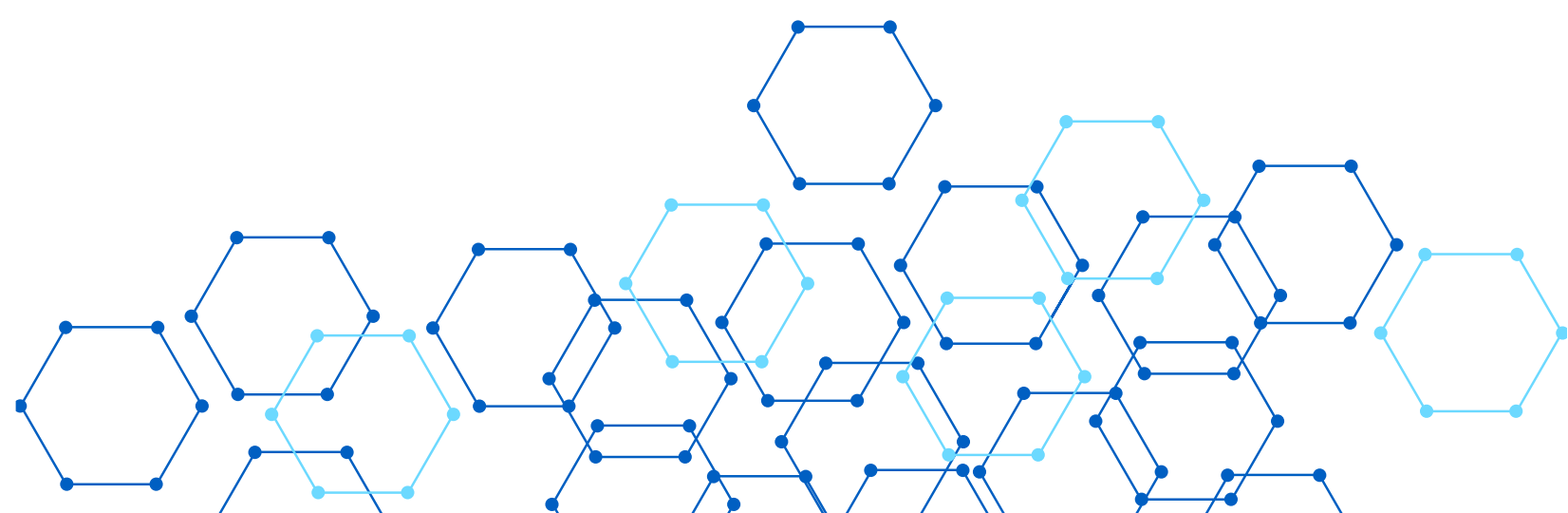


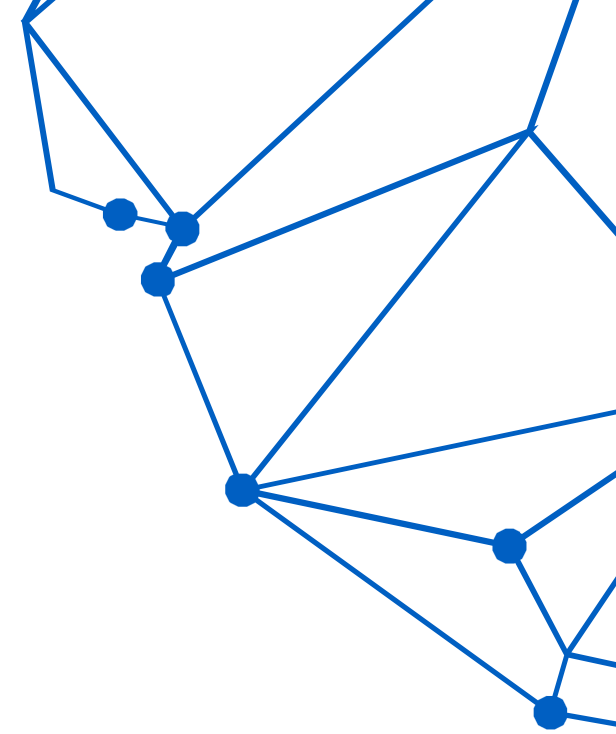
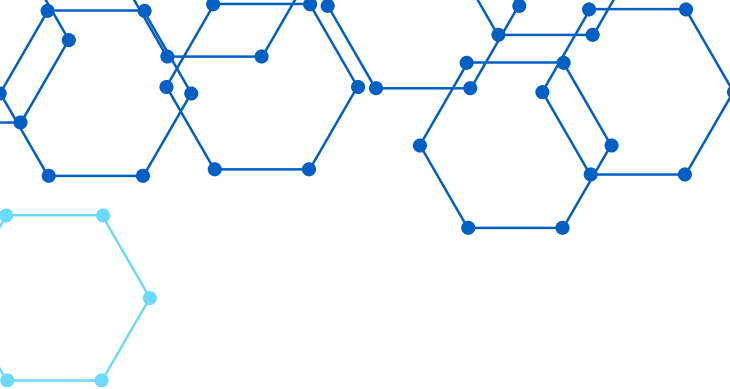
10. WHAT IS THE MAIN DIFFERENCE BETWEEN TESTING AND DEBUGGING?

- A. Testing is done by managers; Debugging is done by developers.
 - B. Testing fixes errors; Debugging finds them.
 - C. Testing establishes the existence of defects; Debugging locates and corrects them.
 - D. Testing happens only after the software is released.
- 
- 

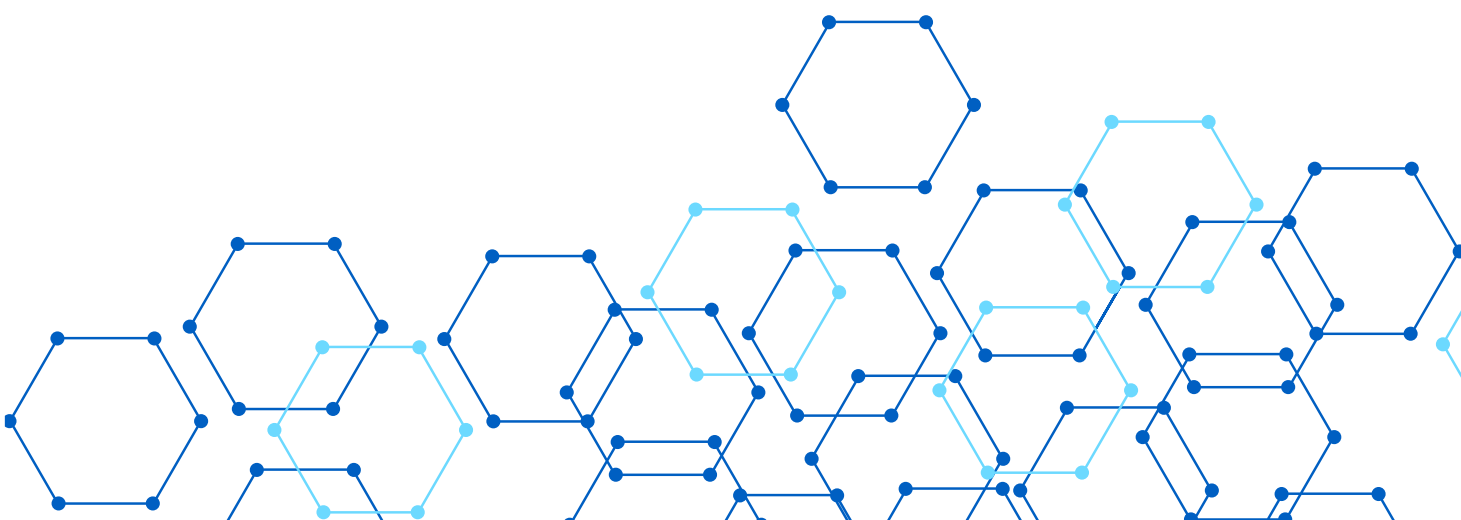
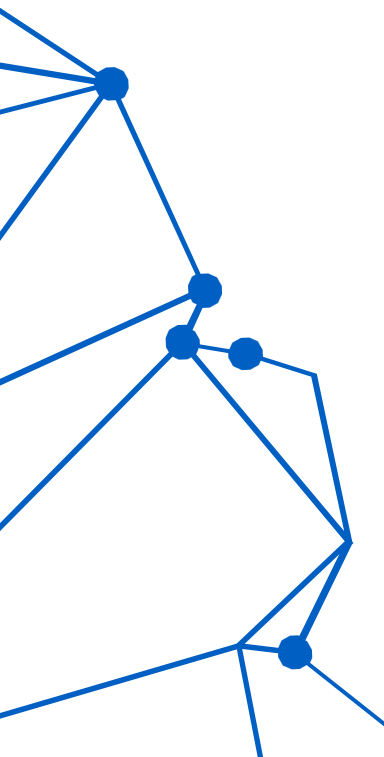


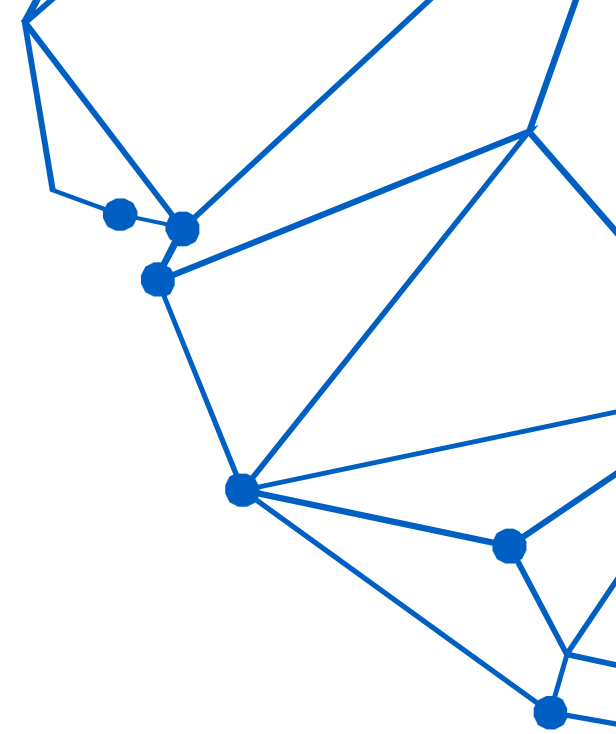
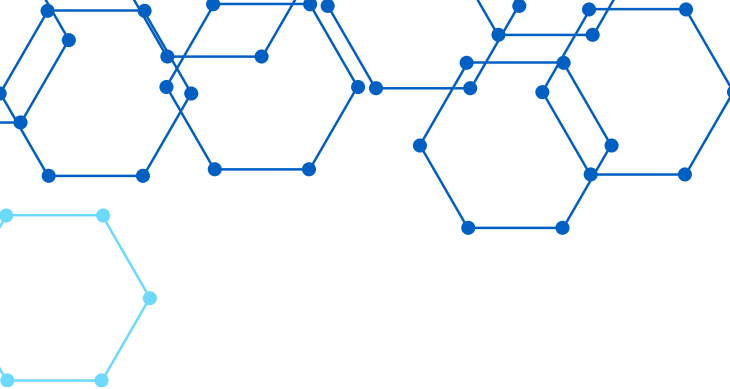
**TRUE OF
FALSE**



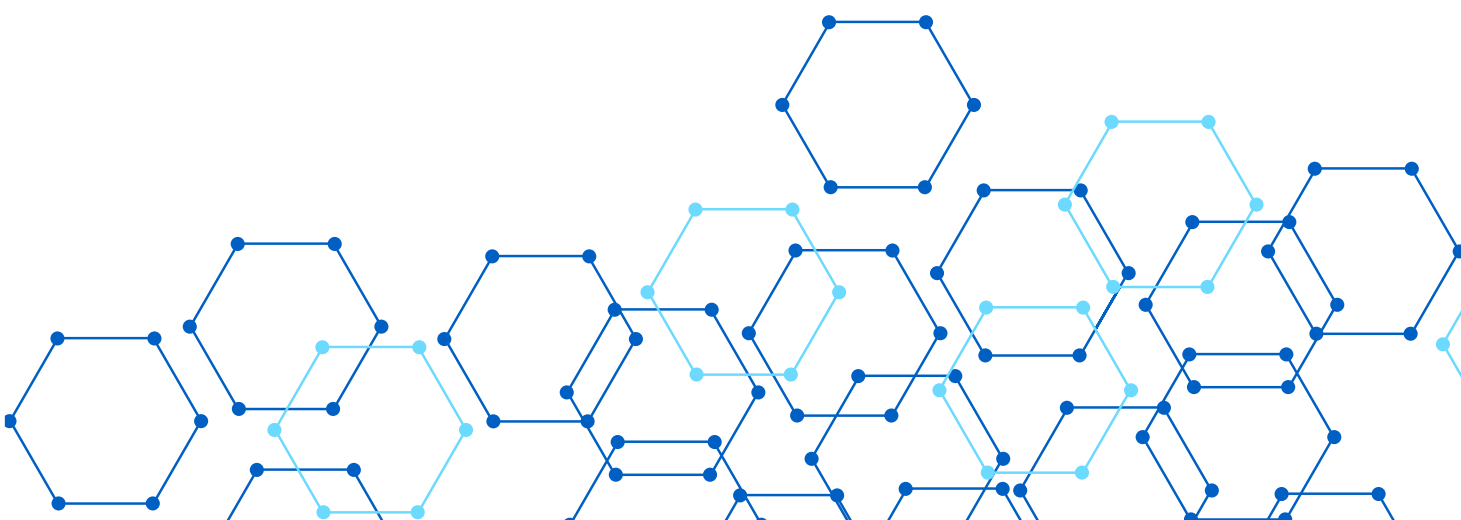
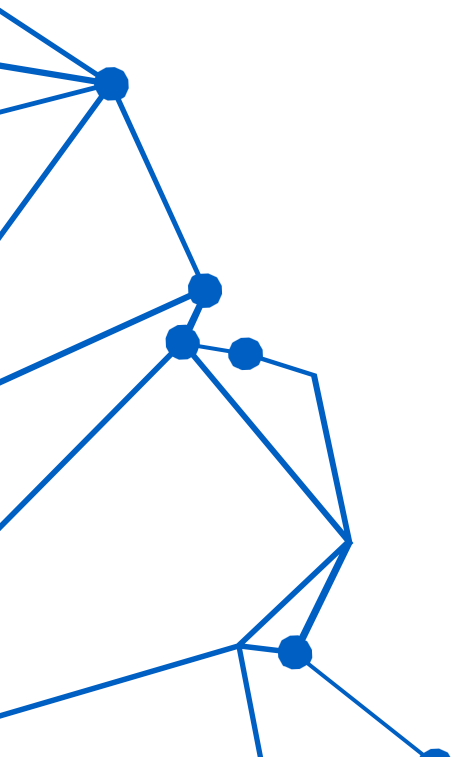


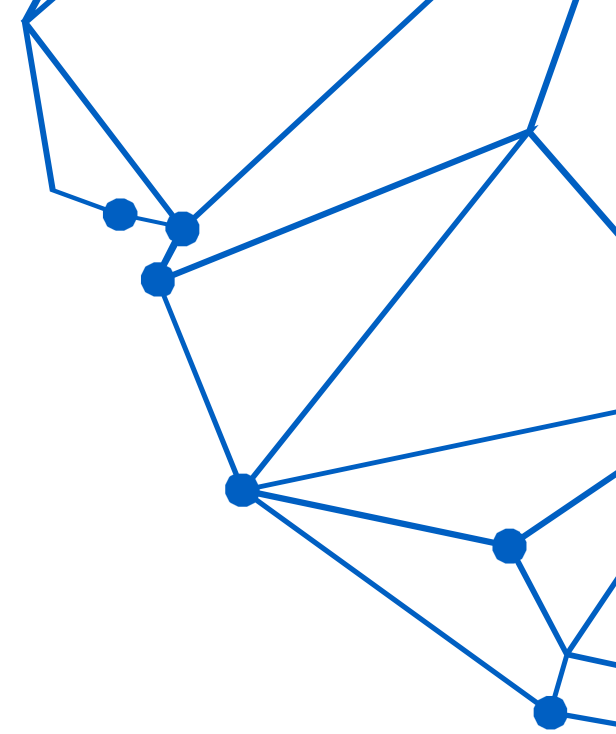
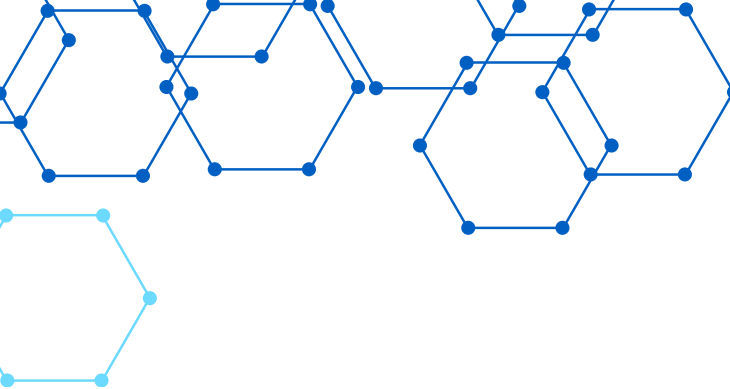
1. THE SOFTWARE PROCESS SERVES AS A ROADMAP FROM REQUIREMENTS GATHERING TO DEPLOYMENT.



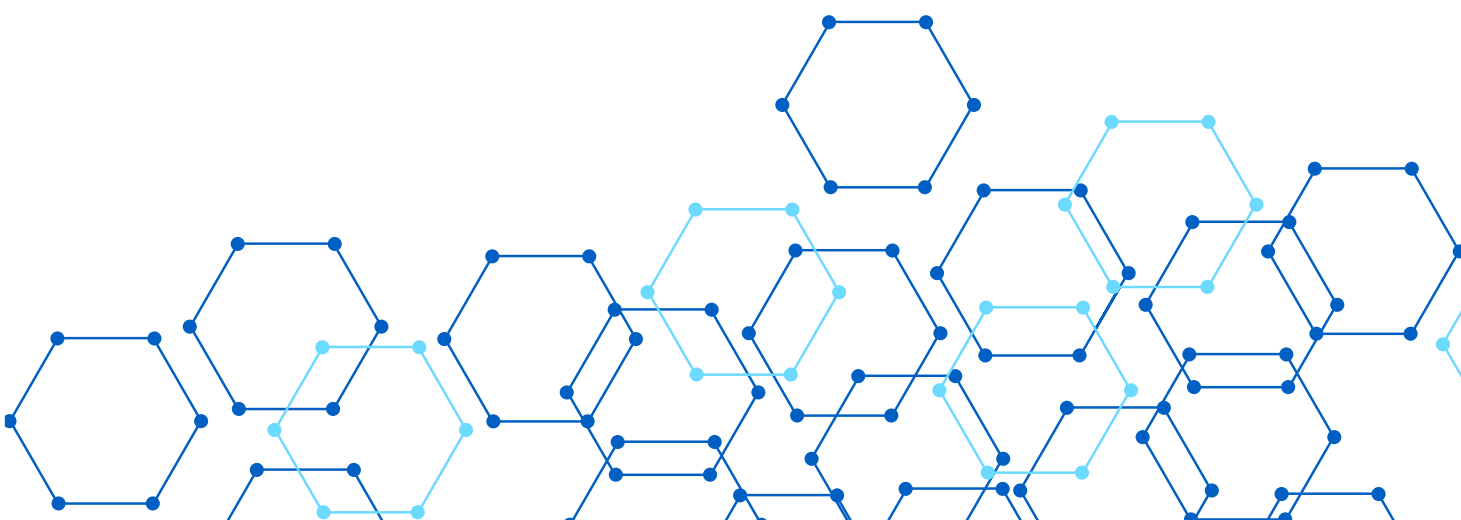
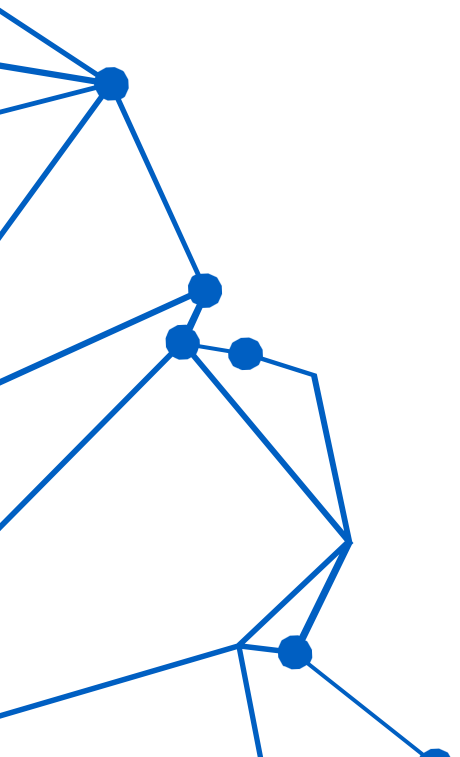


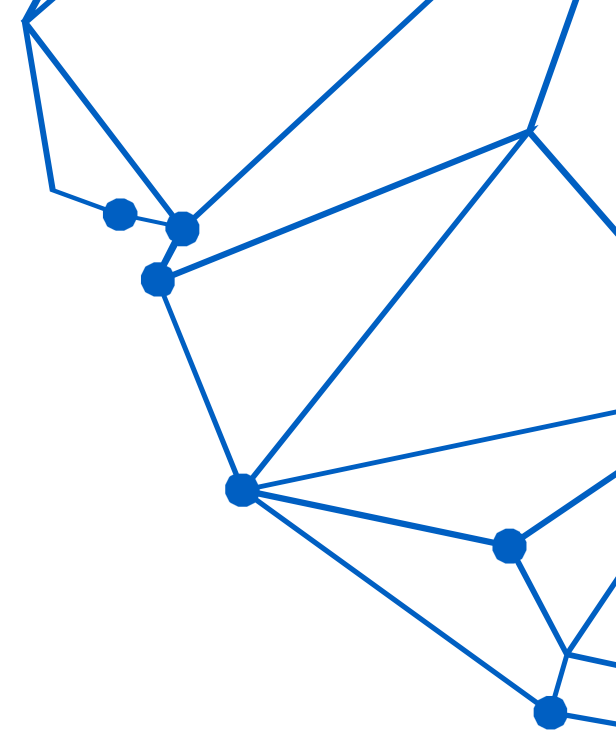
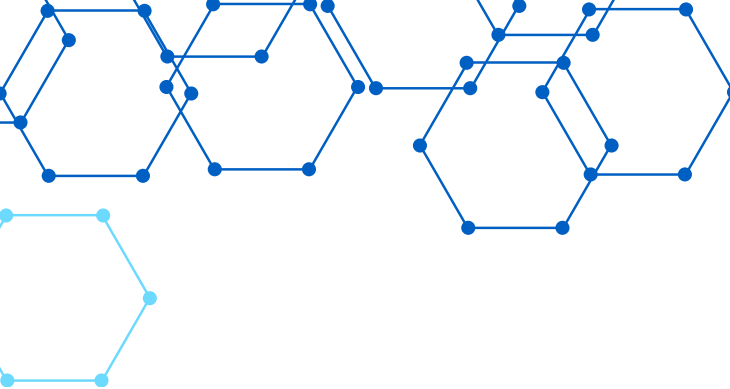
2. AGILE MODELS ARE HIGHLY RIGID AND DO NOT RESPOND WELL TO CHANGE.



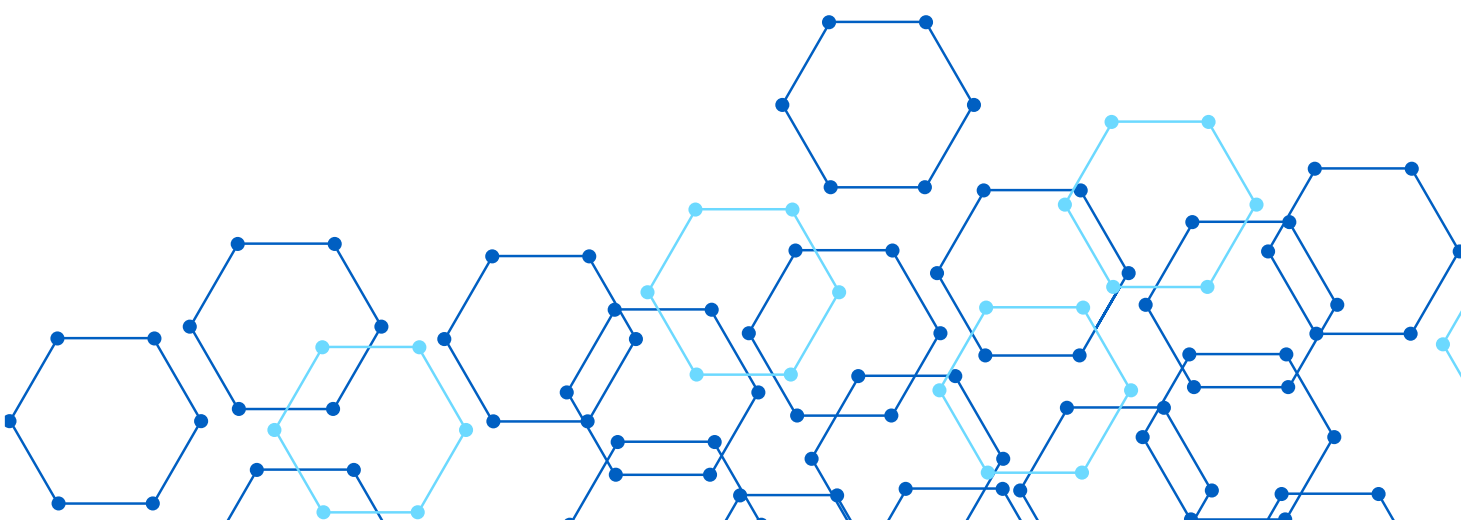
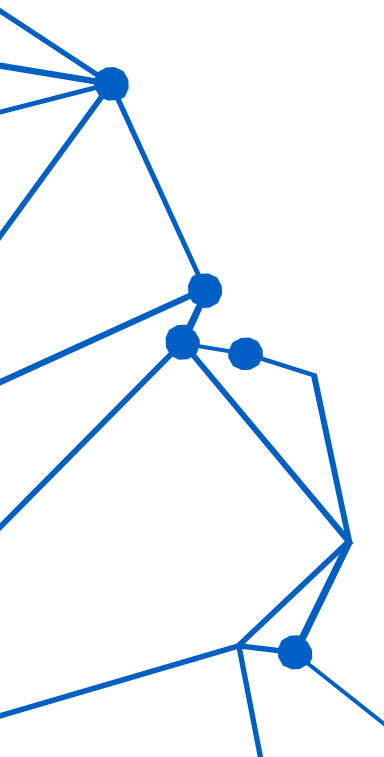


**3. IN THE WATERFALL MODEL, PROCESS
ACTIVITIES ARE ORGANIZED AS
INTERLEAVED STEPS.**

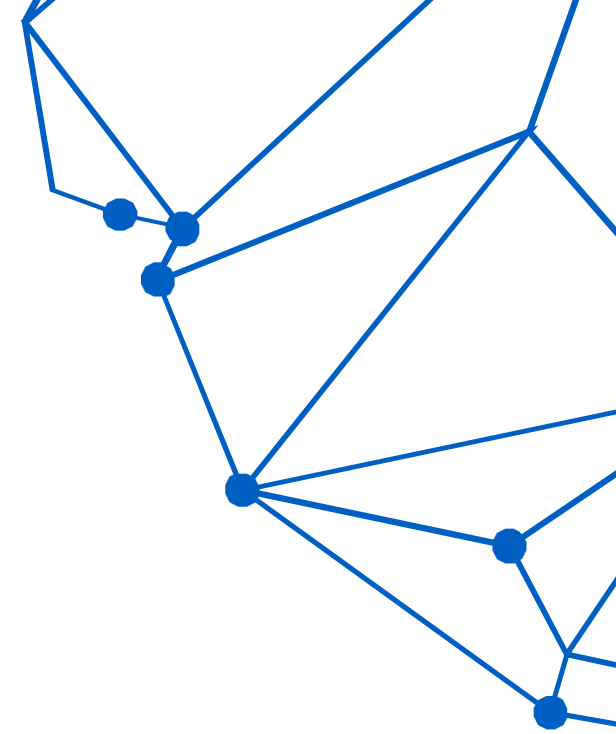
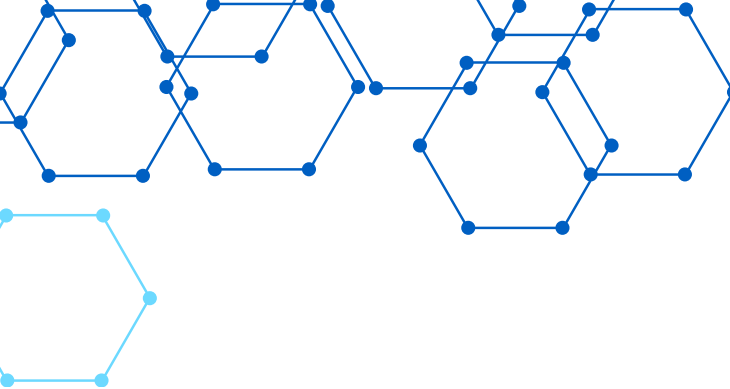




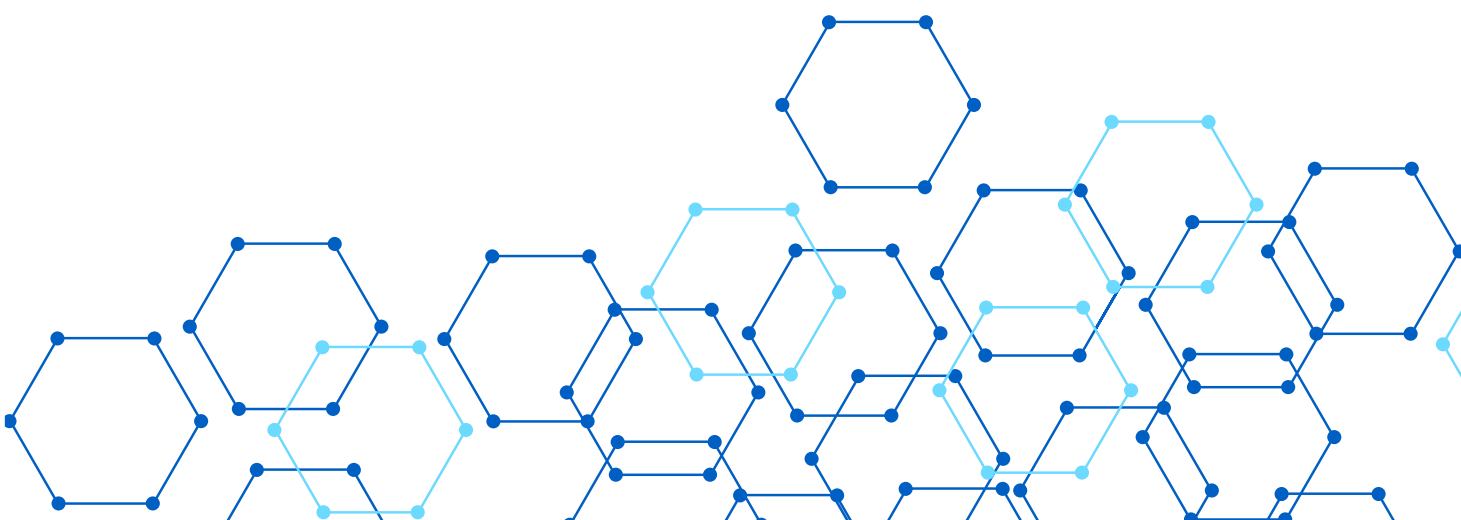
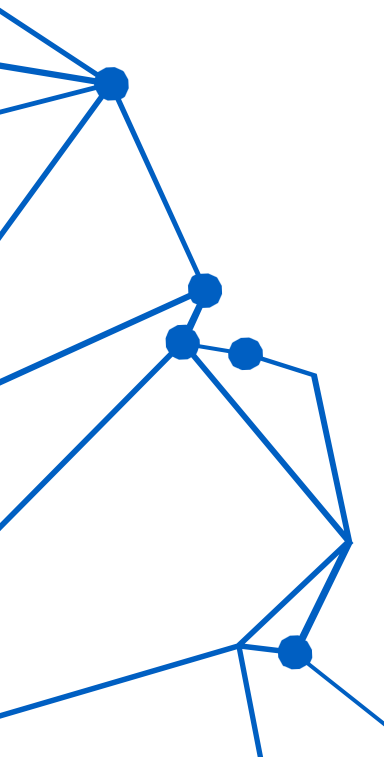
**4. SOFTWARE VALIDATION ENSURES THE
SYSTEM CONFORMS TO ITS
SPECIFICATION AND MEETS CUSTOMER
EXPECTATIONS.**

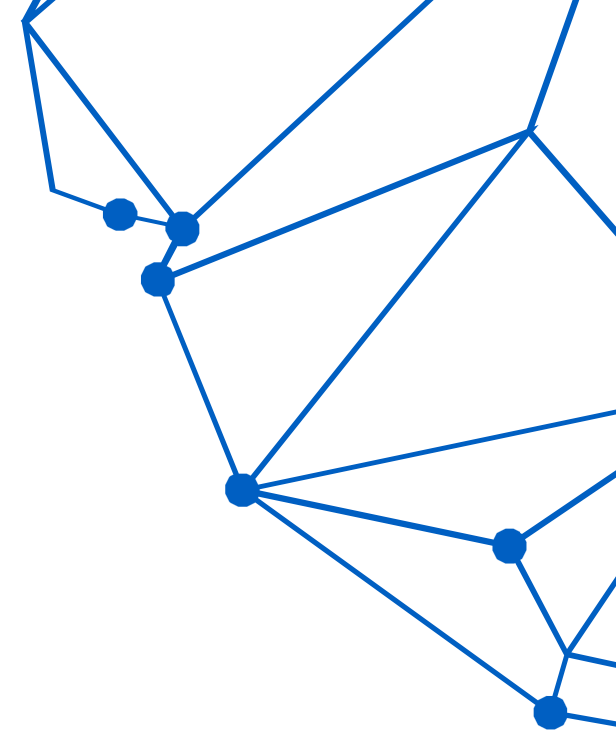
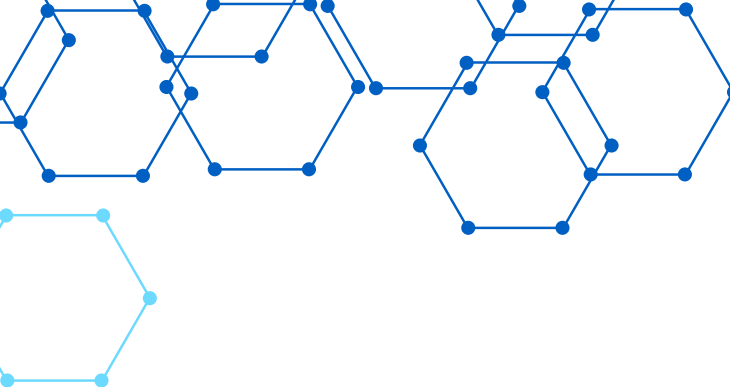


**5. THE ELABORATION PHASE OF RUP
FOCUSES ON DEFINING AND VALIDATING
THE SYSTEM ARCHITECTURE.**

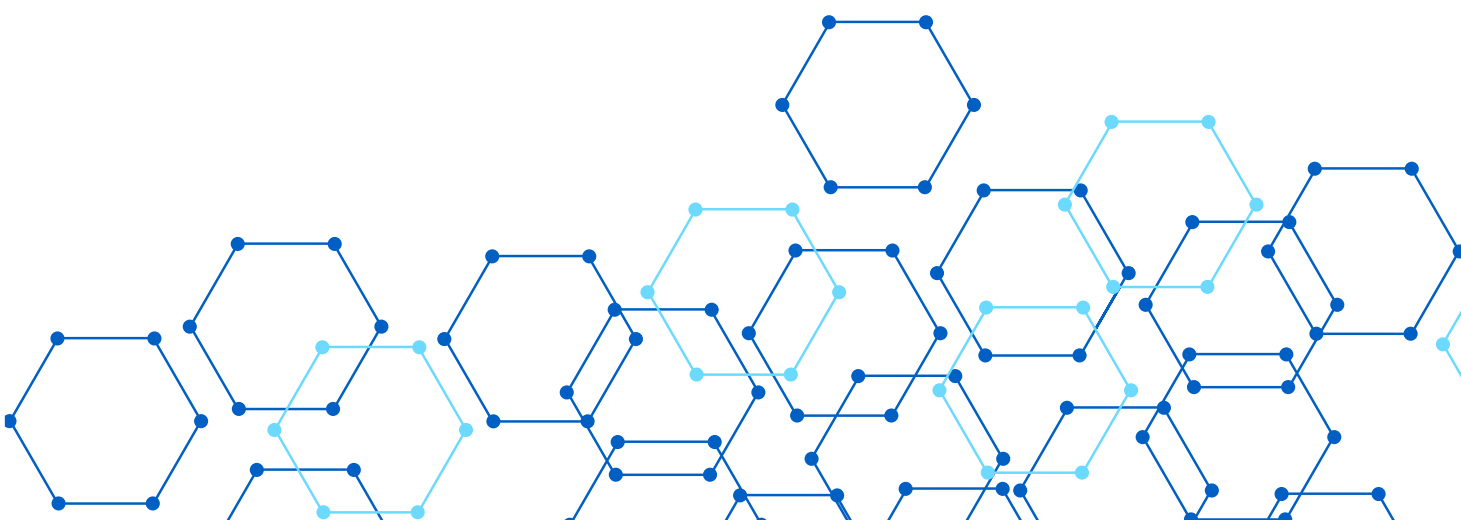
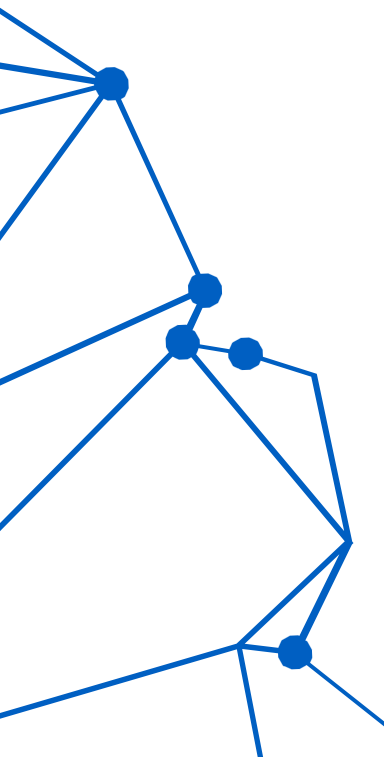


**6. DEBUGGING IS CONSIDERED A PART OF
BOTH THE DEVELOPMENT AND TESTING
STAGES.**

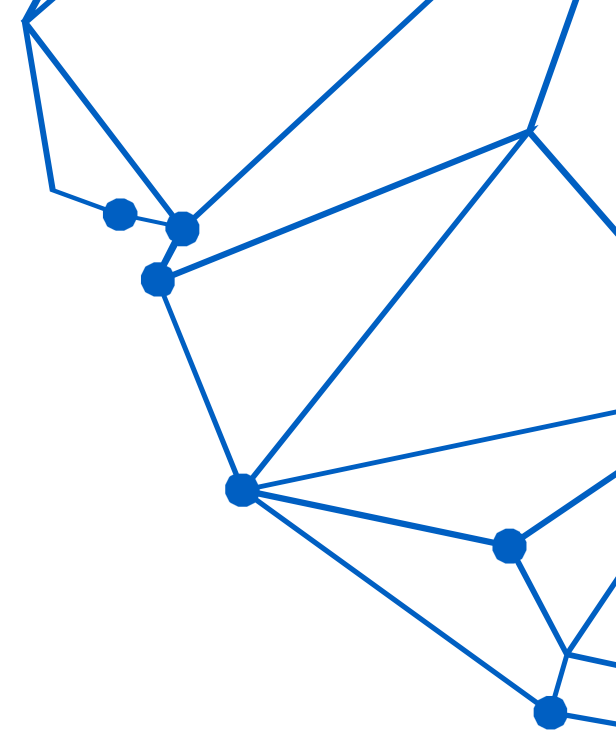
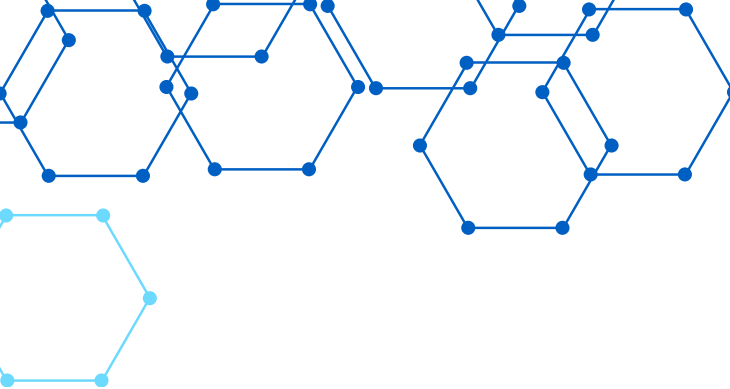




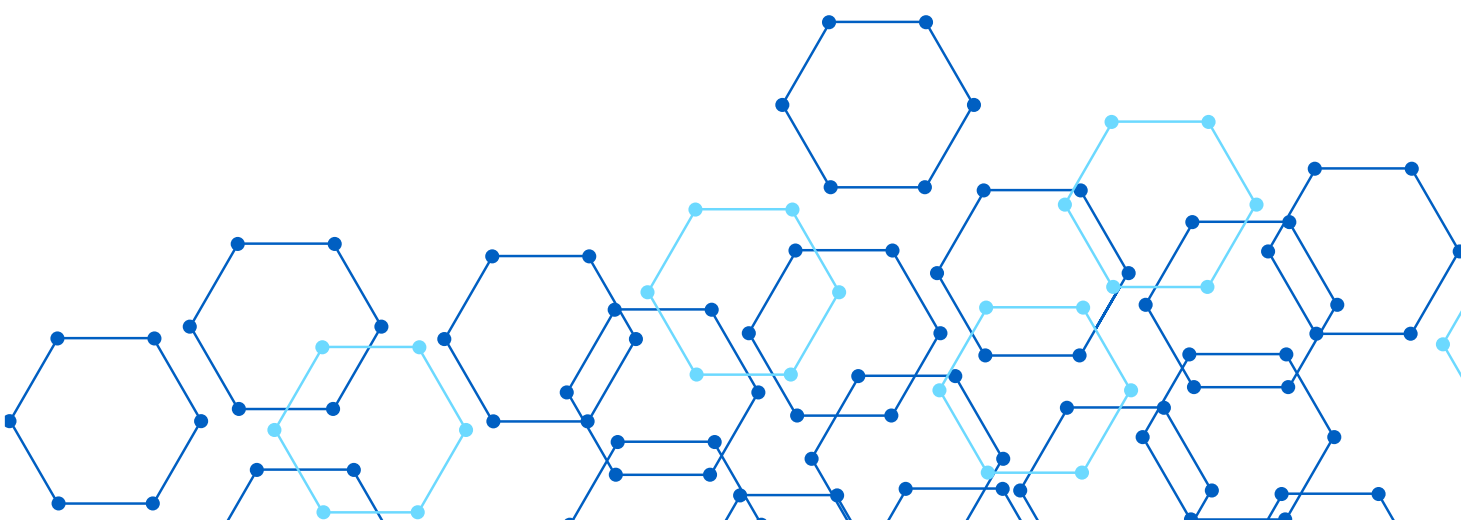
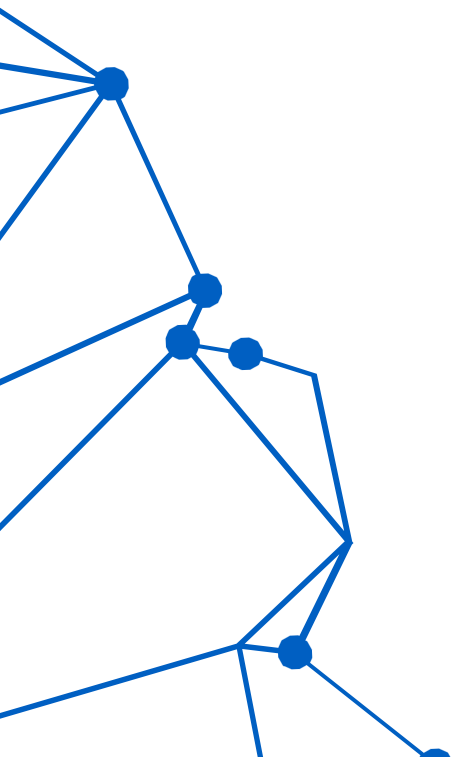
**7. SOFTWARE EVOLUTION REFERS TO
UPDATING A SYSTEM ONLY BEFORE ITS
INITIAL RELEASE.**

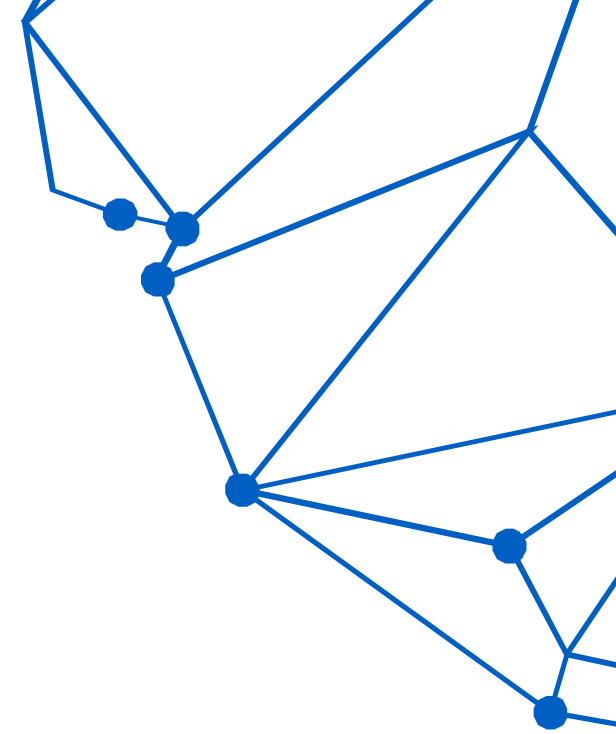
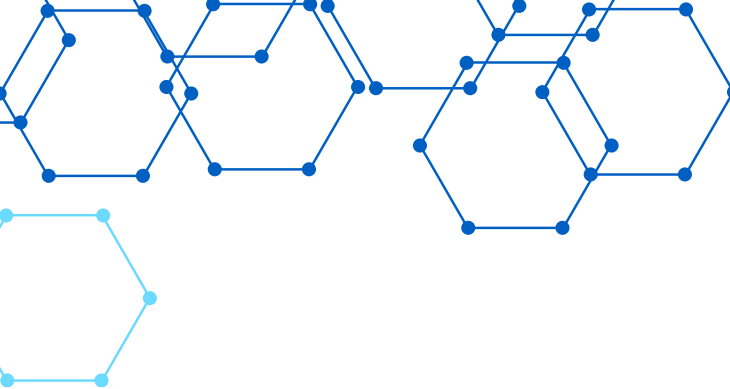


**8. REQUIREMENTS ENGINEERING INCLUDES
GATHERING USER NEEDS THROUGH
ELICITATION AND ANALYSIS.**



**9. RUP IS "ARCHITECTURE-CENTRIC,"
MEANING IT FOCUSES ON SOLID
ARCHITECTURE EARLY IN THE PROJECT.**





**10. HIGH-RISK PROJECTS WITH HIGH
UNCERTAINTY SHOULD IDEALLY USE
THE WATERFALL MODEL.**

